GoodGraph

Методичка по научной графике на Python

Принципы · нормативы · примеры «плохо/хорошо»

Версия 0.1 · ноябрь 2026

Содержание

Раздел 1. Принципы качественной научной графики 3
Раздел 2. Python-инструментарий и настройка среды 14
Раздел 3. Требования к научной графике: цвет, шрифты, размеры и форматы 26
Раздел 4. Основные виды научных графиков и области применения 35
5.01 Линейный график: динамика во времени 41
5.02 Ступенчатый график: события и дискретные изменения 45
5.03 Area chart: площадь как величина 49

Раздел 1. Принципы качественной научной графики

График в научной работе — не украшение и не «картинка к тексту». Это способ рассуждения о данных: читатель должен за несколько секунд увидеть закономерность, оценить её величину и понять, насколько ей можно доверять. Поэтому разговор о графике начинается не с библиотек и не с цветов, а с вопроса: **что именно читатель должен понять и какое решение принять?**

Этот раздел задаёт систему координат для всей методички. Сначала — смысловые принципы: честность, контекст, неопределённость, иерархия. Затем — нормативы и оформление: размеры, шрифты, цвет, единицы, подписи. В конце — разбор одного и того же результата в двух версиях: «плохой» и «правильной».

> Все данные в примерах синтетические. Они правдоподобны по единицам и величинам, но не описывают реальных пациентов.

1.1 «Правильный» и «качественный»: два разных требования

Полезно развести два прилагательных, которые часто используют как синонимы.

Правильный график (valid, truthful) — тот, который не искажает данные и не нарушает нормы:

- масштаб осей соответствует типу кодирования;
- статистика показана корректно (есть *n*, разброс, определение ошибок);
- единицы измерения указаны;
- подпись самодостаточна;
- рисунок соответствует требованиям журнала или ГОСТ.

Качественный график (effective, craft) — тот, который решает задачу читателя:

- сообщение считывается за несколько секунд;
- визуальная иерархия ведёт взгляд от главного к второстепенному;
- палитра доступна при дальтонизме и в чёрно-белой печати;
- шрифты и размеры комфортны в конечном физическом размере;
- график согласован с остальными рисунками работы;
- результат воспроизводим по данным и коду.

Эти требования независимы. График может быть правильным, но бесполезным: аккуратные столбики с *n*, SD и единицами, по которым невозможно ничего сравнить. И наоборот: эффектный график с градиентами и тенями может быть красивым, но лживым. Методичка — о том, как удержать оба требования одновременно.

Главный тезис. График проектируется от вопроса читателя и структуры данных, а не от возможностей библиотеки. Matplotlib умеет рисовать 3D-пончики; это не аргумент в пользу 3D-пончиков.

1.2 Смысловые принципы

1.2.1 Сначала сообщение, потом график

Перед первой строкой кода сформулируйте одно предложение:

> «После этого графика читатель должен понять, что препарат B снижает САД примерно вдвое сильнее плацебо, и что разброс между пациентами велик».

Если предложение не формулируется, график, скорее всего, не нужен: данные лучше показать таблицей. Если предложений два, это два графика или две панели A и B.

Из этого следуют практические правила:

- **Заголовок — вывод, а не тема.** «Результаты исследования» сообщает ноль информации. «Препарат B снижает САД вдвое сильнее плацебо» — сообщает вывод и заодно объясняет, что читать на графике.
- **Один график — одна мысль.** Если хочется показать и динамику, и сравнение групп, и корреляцию, — это три панели, а не один «комбайн».
- **Главное — крупнее и контрастнее.** Основная серия или эффект должны визуально доминировать; вспомогательные — уходить на второй план.
- **Читатель не обязан знать контекст.** График должен работать вместе с подписью, но не вместо неё.

Заголовок внутри рисунка — спорная территория. Для слайдов, отчётов и внутренних документов он полезен. Для журнальной публикации многие редакции предпочитают не дублировать заголовок в изобра-

жении и в подписи: содержательный вывод переносят в caption. Проверяйте гайд журнала; в этой методичке заголовок оставлен как дидактический приём.

1.2.2 Форму определяют данные

Тип графика — не вкусовое решение. Он определяется:

- типом переменных (категориальные, непрерывные, временные, парные);
- числом наблюдений и групп;
- наличием распределения, выбросов, цензурирования;
- задачей: сравнить, показать связь, распределение, динамику, состав.

Минимальная таблица соответствия:

Задача	Рекомендуемая форма	Почему
Сравнить несколько категорий	точка с доверительным интервалом, полоса, столбцы	положение читается точнее длины
Показать распределение	гистограмма, ECDF, box, violin, strip	виден разброс и форма, а не только среднее
Связь двух непрерывных	диаграмма рассеяния, hexbin, contour	видна форма зависимости и выбросы
Динамика во времени	линия; small multiples для многих серий	время должно идти по оси X
Доля целого	stacked bar, 100 % bar; круговая — только для 3–5 долей	углы и площади читаются хуже длины
Матрица значений	heatmap с ограниченной палитрой	цвет кодирует величину, но требует шкалы
Парные измерения	slope chart, paired dot plot	видно направление изменения у каждого объекта
Оценки и неопределённость	forest plot	компактно показывает эффект и ДИ
Выживаемость	Kaplan–Meier с числом рисков	цензурирование нельзя игнорировать
Диагностическая точность	ROC-кривая, precision–recall	зависит от prevalence
Многомерные наблюдения	small multiples, PCA/biplot (осторожно)	уменьшение размерности меняет интерпретацию

Подробный разбор видов графиков — в разделе 4, примеры с кодом — в разделе 5.

Полезно помнить «грамматику графики» (Wilkinson, 1999; ggplot2): данные → эстетические отображения → геометрия → шкалы → грани. Matplotlib мыслит императивно («нарисуй линию здесь»), ggplot2 и plotnine — декларативно («свяжи x и y, сгруппируй по цвету»). Оба подхода допустимы; декларативный лучше защищает от несогласованности, императивный — даёт полный контроль.

1.2.3 Честность масштаба

Это самый частый источник визуальной лжи — и самый простой для исправления.

Столбцы кодируют длину. Длина обязана начинаться с нуля. Если ось Y обрезана, читатель сравнивает не значения, а обрезки. Пример из раздела 1.4: средние 4,2 и 9,0 мм рт. ст., ось начинается с 3,5. Визуально препарат B «в 7,5 раза больше» плацебо, хотя в данных разница в 2,1 раза. Коэффициент искажения (lie factor, Tufte):

> **LF** = (5,46 / 0,73) / (8,96 / 4,23) ≈ 3,5.

Значения LF заметно выше 1 или заметно ниже 1 означают искажение; Tufte предлагал считать допустимым диапазон примерно 0,95–1,05.

Точки и линии кодируют положение. Для них нулевая отметка не обязательна: если весь смысл в изменении на 2–3 единицы, растянутая ось помогает увидеть тренд. Но у этого есть условия:

- ось явно подписана, диапазон не скрывает важный контекст;
- читатель не принят за дурака: обрезка не маскирует выбросы и не создаёт ложный «обвал» или «взлёт»;
- для сравнения нескольких серий шкалы одинаковы, если это возможно.

Логарифмическая шкала допустима для величин, которые меняются мультипликативно (титры антител, вирусная нагрузка, концентрации), и только для положительных значений. Она обязана быть подписана: «лог. шкала, основание 10». Логарифм не «улучшает» график — он меняет вопрос, на который тот отвечает.

Разрыв оси допустим как крайняя мера для выбросов, с явными метками разрыва. Для столбцов — недопустим: он ломает длину. **Две оси Y** на одном графике почти всегда создают ложную корреляцию: две произвольно масштабированные величины всегда можно «подогнать» так, чтобы линии пересеклись в нужном месте. Если без этого нельзя обойтись, кодируйте оси цветом, подписывайте единицы и объясняйте, почему две панели хуже.

Пропорции. Для временных рядов удачный приём — «banking to 45°»: выбирать соотношение сторон так, чтобы типичный наклон тренда был около 45°. Слишком широкий график сглаживает изменения, слишком узкий — драматизирует. Для сравнения наклонов в scatter-plot полезна квадратная область.

1.2.4 Неопределённость, распределение и *n*

Столбик со средним и «усиками» — не универсальный график. У него есть презрительное, но точное название — *dynamite plot*: он скрывает форму распределения, число наблюдений и выбросы. Weissgerber и соавторы (2015) показали, как часто такие графики искажают картину в биомедицинских статьях.

Что показывать:

- ***n* для каждой группы** — в подписи или на оси.
- **Разброс**: отдельные точки при небольшом *n*, box/violin или strip при большем, SD/CI — как дополнение, а не замена.
- **Определение ошибок**: SD, SEM и 95 % ДИ — разные вещи.

Обозначение	Что характеризует	Как читать
SD	разброс наблюдений вокруг среднего	«насколько разные пациенты»
SEM	точность выборочного среднего, SD/√*n*	«насколько точно измерено среднее», не разброс
95 % ДИ	неопределённость среднего	при повторных выборках накрывал бы истинное среднее в 95 % случаев

Частая ошибка — подписать «mean ± SEM», а в тексте интерпретировать как «разброс». Ещё чаще — использовать 95 % ДИ как «диапазон, в котором лежат 95 % данных»: это неверно, для этого нужен интервал предсказания или перцентили.

Для парных измерений (до/после у одного пациента) столбики независимых групп теряют связь между наблюдениями. Показывайте пары: slope chart или paired dot plot. Для скошенных данных — медиана и межквартильный интервал, а не среднее и SD.

И наконец: **эффект и его неопределённость важнее p-значения**. «$p < 0{,}05$» не говорит, насколько велик эффект и насколько он клинически значим. Показывайте разницу средних, отношение рисков, hazard ratio — с 95 % ДИ.

1.2.5 Контекст и воспроизводимость

График без контекста — головоломка. Минимальный набор:

- единицы измерения на осях и в подписи;
- *n* и способ формирования групп;
- что означают ошибки/полосы;
- статистический тест или модель, если они упомянуты;
- программное обеспечение и версия (для воспроизводимости);
- доступность данных и кода, если это допускает журнал.

Дополнительно:

- **Согласованность групп.** Один и тот же цвет и порядок групп во всех рисунках статьи. Читатель не должен каждый раз заново расшифровывать легенду.
- **Нормализация.** «Относительно контроля» — относительно какого контроля, что именно делили, что делать с нулями? Всё это — в подпись.
- **Пропуски.** Если данные исключены, покажите сколько и почему.
- **Версии.** «Построено в Python 3.12, matplotlib 3.11, seaborn 0.13» — дешёвая и полезная строка в подписи или supplementary.

1.2.6 Иерархия, минимализм и декор

Эдвард Тафти ввёл два полезных понятия: **data-ink ratio** (доля «чернил», которая кодирует данные) и **chartjunk** (декор, не несущий информации). Практический вывод: убирайте всё, что не помогает читать данные.

Что обычно лишнее:

- тени, псевдо-3D, градиенты, не кодирующие величину;
- тяжёлая сетка по обеим осям и полная рамка;
- легенда, дублирующая подписи оси;
- декоративные иконки, фоны, текстуры;
- подписи «Рисунок 1» внутри изображения;
- несколько акцентных цветов без семантики.

Но минимализм — не самоцель. Убрать единицы, *n* или подпись оси — это не минимализм, а потеря информации. Хороший график не «пустой», а **иерархичный**:

1. данные — самый контрастный слой;
2. оси, подписи, аннотации — средний;
3. сетка, рамки, фон — самый бледный.

Полезные приёмы:

- **прямые подписи** вместо легенды, если серий немного;
- **один акцентный цвет** для главного вывода, серый — для контекста;
- **лёгкая сетка** только по той оси, где она помогает читать значения;
- **белое пространство** как способ группировки, а не «пустота».

1.2.7 Доступность

Около 8 % мужчин европейского происхождения имеют нарушения цветового зрения (протанопия или дейтеранопия); среди женщин доля ниже, но не нулевая. В зале на 250 человек, по оценке Color Universal Design, может быть больше десяти человек с дальтонизмом. Рецензент тоже может быть одним из них.

Правила:

- **Не кодировать смысл только цветом.** Дублируйте формой маркера, типом линии, прямой подписью, штриховкой.
- **Использовать палитры, проверенные на дальтонизм**: Okabe–Ito, Paul Tol, viridis/cividis для последовательных шкал.
- **Проверять в оттенках серого.** Если после обесцвечивания серии неразличимы — добавить форму или подпись.
- **Контраст.** Для текста — не ниже 4,5:1, для графических объектов — не ниже 3:1 (WCAG 2.1).
- **Кегль в конечном размере.** Шрифт 6 pt, который «нормально смотрится» на экране в 200 %, в журнале превратится в нечитаемую полосу.
- **Alt-текст** для веб-версий и доступных PDF.

1.2.8 Согласованность

Серия рисунков в статье должна выглядеть как один документ, а не как коллаж из разных статей. Единый визуальный язык:

- одна гарнитура и 2–3 кегля;
- одна палитра и одно соответствие «цвет → группа»;
- одинаковые толщины линий, размеры маркеров, стиль сетки;
- одинаковое расположение меток панелей (A, B, C) — слева сверху;
- одинаковый десятичный разделитель и формат дат;
- одинаковые единицы и сокращения.

Если на рисунке 1 плацебо серое, на рисунке 2 оно тоже серое. Если на рисунке 3 появился новый препарат — он получает новый цвет, а не переиспользует цвет старого.

1.2.9 Проверка «пять секунд — тридцать секунд — пять минут»

Быстрый тест перед отправкой:

1. **Пять секунд.** Что читатель понял? Если ответ «какие-то столбики» — график не работает.

2. **Тридцать секунд.** Можно ли прочитать значения, единицы, *n* и понять, что означают усы? Если нет — не хватает подписей.
3. **Пять минут.** Можно ли восстановить, как получены данные, что такое ошибки, есть ли выбросы и не противоречит ли график тексту? Если нет — подпись не самодостаточна.

1.3 Нормативы и оформление

1.3.1 Какие документы задают правила

Иерархия проста: **требования журнала важнее ГОСТ**, если вы отправляете статью. ГОСТ и локальные шаблоны — для диссертаций, отчётов о НИР и внутренних документов.

Документ	Что регулирует	Комментарий
Гайд журнала (Instructions for Authors)	размеры, шрифты, форматы, число рисунков, captions	всегда проверяйте перед отправкой
ICMJE Recommendations	общие нормы медицинских журналов, подготовка рукописи	icmje.org
EQUATOR Network	отчётные стандарты (CONSORT, STROBE, PRISMA, ARRIVE)	equator-network.org
COPE	публикационная этика, целостность изображений	publicationethics.org
ГОСТ 7.32-2017	отчёт о НИР: структура, рисунки, подписи	docs.cntd.ru
ГОСТ 2.105-95	ЕСКД: общие требования к текстовым документам	docs.cntd.ru
ГОСТ 8.417-2002	единицы величин	docs.cntd.ru

Отчётные стандарты важны не только для текста. CONSORT требует схему потока участников, STROBE — аккуратного описания выборки, PRISMA — блок-схему отбора исследований, ARRIVE — деталей экспериментов на животных. Это тоже «научная графика», и к ней предъявляются те же требования.

Целостность изображений. Нельзя selectively enhance, замазывать, склеивать фрагменты без явных границ, менять яркость так, чтобы исчезали детали. Любая обработка — в подписи или supplementary. Это прямая норма COPE и ICMJE, а не «стилистика».

1.3.2 Размер, разрешение, формат

Проектируйте в конечном размере. Если рисунок будет шириной 85 мм, не рисуйте его в 16 дюймов и не уменьшайте в Word: шрифты и толщины превратятся в лотерею.

Параметр	Типичное требование	Комментарий
Ширина	85–90 мм (1 колонка), 170–183 мм (2 колонки)	Nature: 89 и 183 мм
Шрифт	sans-serif, 7–10 pt в конечном размере	нижняя граница 5–7 pt у отдельных журналов
Линии	0,25–1 pt	тоньше 0,25 pt может исчезнуть при печати
Растр	300 dpi для фото, 600–1200 dpi для штриховых	TIFF/PNG, без JPEG для линий
Вектор	PDF, EPS, SVG, AI	предпочтительно для графиков и схем
Цвет	RGB/sRGB для онлайн, CMYK для печати	конвертировать в конце, проверять gamut
Поля	3–5 мм	чтобы подписи не обрезались

Практические следствия:

- **Не скриншот.** Экспорт из Python, R, GraphPad, Origin — но не снимок экрана: скриншот теряет вектор и разрешение.
- **Шрифты — в кривые или встраивать.** Для SVG удобно `svg.fonttype = "path"`; для PDF — `pdf.fonttype = 42`, чтобы текст оставался редактируемым, но не подменялся при просмотре.
- **Не масштабировать непропорционально.** Растянутый по ширине график меняет восприятие наклонов и пропорций.
- **Проверять в 100 %.** Откройте итоговый PDF при реальном размере и посмотрите, читаются ли деления и легенда.

1.3.3 Шрифты

Для рисунков почти всегда лучше **sans-serif**: Arial, Helvetica, Liberation Sans, DejaVu Sans. Они плотнее и лучше читаются в мелком кегле, чем засечные гарнитуры.

Элемент	Ориентир
Заголовок внутри рисунка	9–10 pt
Подписи осей	8–9 pt
Деления и подписи делений	7–8 pt
Легенда	7–8 pt
Подпись к рисунку (caption)	8–9 pt, обычно в тексте, не в изображении

Правила:

- одна гарнитура на весь рисунок и, желательно, на всю статью;
- не писать подписи КАПСОМ — это замедляет чтение;
- не делать всё жирным: жирный — только для акцента;
- не использовать тонкие начертания (Light, Thin) в мелком кегле;
- проверить кириллицу и специальные символы: μ, ±, ≥, ≤, →, °C;
- математические переменные — курсивом (*p*, *n*, *M*), единицы — прямым;
- не смешивать в одном рисунке разные семейства шрифтов без причины.

1.3.4 Цвет

Цвет должен **кодировать смысл**, а не оживлять картинку.

Три типа палитр:

- **качественная** — для категорий (группы, препараты, регионы);
- **последовательная** — для упорядоченных значений (концентрация, время);
- **дивергентная** — для отклонений от нуля/нормы (изменение, разница).

Базовый набор, который стоит запомнить, — палитра Okabe–Ito:

Цвет	HEX	Разумная роль
оранжевый	`#E69F00`	категория 1
небесно-голубой	`#56B4E9`	категория 2
сине-зелёный	`#009E73`	категория 3
жёлтый	`#F0E442`	категория 4 (только с тёмным контуром)
синий	`#0072B2`	основной результат
вермилион	`#D55E00`	акцент/второй результат
красновато-фиолетовый	`#CC79A7`	категория 5
чёрный	`#000000`	текст, оси, а не серии

Правила:

- не больше 6–8 категориальных цветов; больше — объединяйте или делайте small multiples;
- не используйте «радугу» и `jet`: она создаёт ложные границы и не воспринимается равномерно;
- не кодируйте противоположные категории только красным и зелёным;
- серый — для контроля, фона и «прочего»; он не должен конкурировать с данными;
- проверяйте рисунок в градациях серого и через симулятор дальтонизма;
- фон — белый (для печати), без тёмных тем;
- контраст графических объектов — не ниже 3:1.

Для последовательных шкал используйте `viridis`, `cividis`, `Blues`, `Greens`; для дивергентных — `RdBu_r`, `PuOr`, `BrBG` с симметричным диапазоном относительно нуля. Центр дивергентной шкалы должен совпадать с содержательным нулём, а не с серединой диапазона данных.

1.3.5 Единицы, числа, статистика

- **СИ и общепринятые медицинские единицы.** «мм рт. ст.» — внесистемная, но допустимая в медицине единица; «мг/кг», «ммоль/л», «уд/мин».

- **Пробел между числом и единицей:** 8,9 мм рт. ст., 10 мг, 5 %, 37 °C. Исключения — знаки градусов угла и некоторые типографские случаи.
- **Десятичный разделитель.** В русском тексте — запятая, в англоязычных журналах — точка. Главное — единообразие в пределах работы. В коде и CSV обычно точка; в подписях рисунка — по языку публикации.
- **Ведущий ноль:** 0,05, а не ,05.
- **Тысячи:** 12 000, а не 12000, в тексте; в осях можно 12 000 или 12 тыс.
- **Средние:** M ± SD; медиана и IQR: Me [Q1; Q3]; доли: n (%).
- **p-значение:** p = 0,032, а не «p<0.05» без точного значения, если журнал не требует иначе. Курсив для p, n, M.
- **Сокращения** расшифровывайте при первом упоминании в подписи.

1.3.6 Подписи к рисункам

Подпись — не «название файла», а самостоятельный текст. Хорошая структура:

1. **Что показано** — одно предложение с выводом или темой.
2. **Как показано** — группы, n, что означают точки/столбцы/усы.
3. **Определения** — ошибки, сокращения, статистический тест.
4. **Источник/ПО** — данные, код, версия, при необходимости.

Плохо:

> Рисунок 1. Результаты.

Хорошо:

> **Рисунок 1.** Снижение систолического АД через 12 недель терапии. Точки — индивидуальные значения пациентов, маркеры — среднее, усы — 95 % доверительный интервал (t-распределение). Плацебо — n = 48, препарат А 10 мг — n = 51, препарат В 20 мг — n = 49. Штриховая линия — отсутствие изменения. Данные синтетические; код — `01_principles/scripts/good_plot.py`.

Формальные детали:

- в ГОСТ 7.32 подпись обычно начинается с «Рисунок 1 — Название» (с тире);
- в англоязычных журналах — «Figure 1. …»;
- подпись располагается под рисунком, а не внутри изображения;
- точка в конце подписи — по стилю журнала;
- ссылка в тексте — до появления рисунка: «(рисунок 1)» или «(Fig. 1)».

1.3.7 Чек-лист перед отправкой

- ☐ Заголовок/подпись сообщают вывод, а не «результаты».
- ☐ Тип графика соответствует задаче и структуре данных.
- ☐ Столбцы начинаются с нуля; обрезка осей объяснена.
- ☐ Логарифмическая шкала подписана.
- ☐ Нет двух осей Y без крайней необходимости.
- ☐ Показаны n, разброс и определение ошибок.
- ☐ Единицы указаны на осях и в подписи.
- ☐ Цвета различимы при дальтонизме и в градациях серого.
- ☐ Шрифт ≥ 7 pt в конечном размере; кириллица и символы не потеряны.
- ☐ Ширина соответствует колонке журнала (85/170 мм или гайд).
- ☐ Вектор или растр ≥ 300 dpi; нет скриншотов.
- ☐ Верхняя и правая рамки, тяжёлая сетка, тени и 3D убраны.
- ☐ Легенда не дублирует подписи и не закрывает данные.
- ☐ Подпись самодостаточна: группы, n, тест, сокращения.
- ☐ Рисунок согласован с другими рисунками по палитре и шрифтам.
- ☐ Нет искажений изображения и необъявленной обработки.
- ☐ Файлы названы по схеме журнала; формат и цветовой профиль проверены.
- ☐ Рисунок проверен в PDF при 100 % масштабе.

1.4 Один результат — два графика

Ниже — один и тот же набор данных: 148 пациентов, три группы, изменение САД через 12 недель. Слева — «плохая» версия, справа — «правильная». Обе построены по одному CSV; различается только код визуализации.

1.4.1 Плохой график

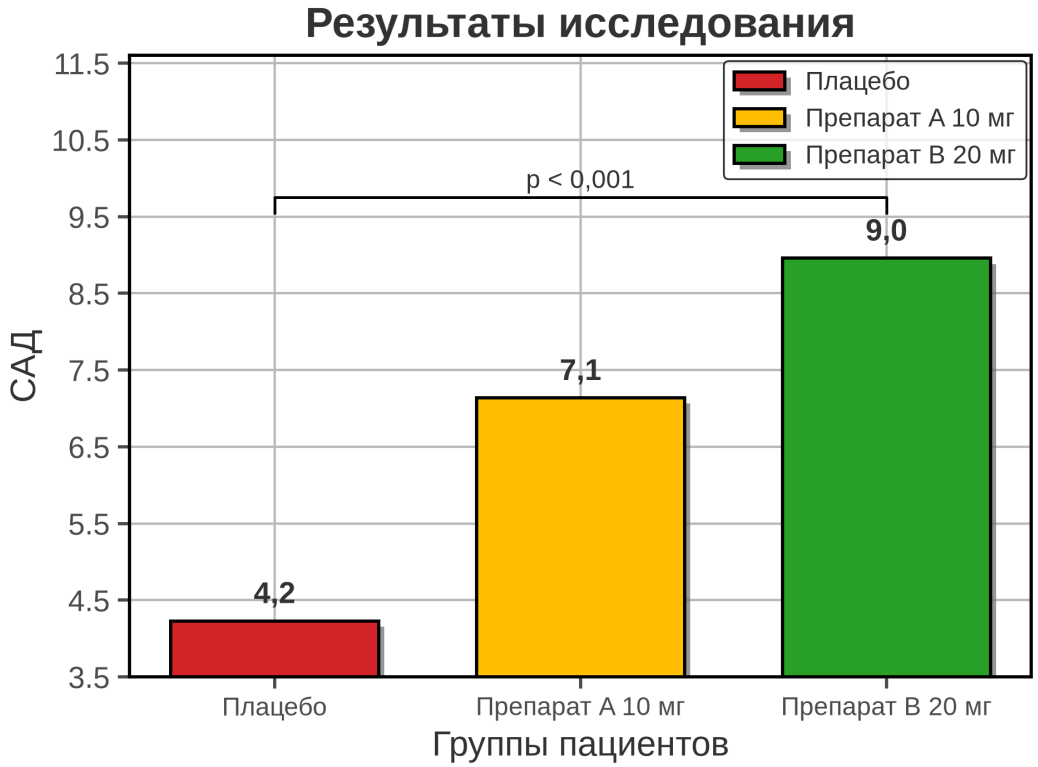


Рисунок 1.1. Плохой график: обрезанная ось, светофорная палитра, тени, тяжёлая сетка и легенда внутри поля данных.

Код: `01_principles/scripts/bad_plot.py`. Данные: `data/00_dataset.csv`.

1.4.2 Что с ним не так

1. **Обрезанная ось.** Столбцы начинаются с 3,5 мм рт. ст., а не с нуля. Визуальное отношение В/плацебо — около 7,5, реальное — 2,1. Lie factor ≈ 3,5.
2. **Светофорная палитра.** Красный — жёлтый — зелёный не только неразличим при протанопии/дейтеранопии, но и навязывает оценку «плохо — средне — хорошо», которой в данных нет.
3. **Тени и псевдо-3D.** Тень не кодирует ни одну переменную; она лишь утяжеляет нижнюю часть столбцов и мешает сравнивать высоту.
4. **Тяжёлая сетка по обеим осям.** Сетка конкурирует с данными; вертикальные линии вообще не помогают читать столбцы.
5. **Полная рамка и жирные линии.** Визуальный шум, который не несёт смысла.
6. **Легенда дублирует подписи.** Названия групп уже написаны под столбцами; легенда занимает место и закрывает часть данных.
7. **Заголовок «Результаты исследования».** Он не сообщает, что именно показано и какой вывод должен сделать читатель.
8. **Нет единиц, *n*, разброса и определения ошибок.** «САД» — это что: исходное, конечное, изменение? В каких единицах? Сколько пациентов?
9. **Только средние.** Без индивидуальных точек и ДИ нельзя понять, насколько велик разброс и насколько устойчива разница.
10. **p < 0,001 без теста и эффекта.** Значимость не заменяет величину эффекта; не указано, какой тест применялся и какие группы сравнивались.

1.4.3 Правильный график

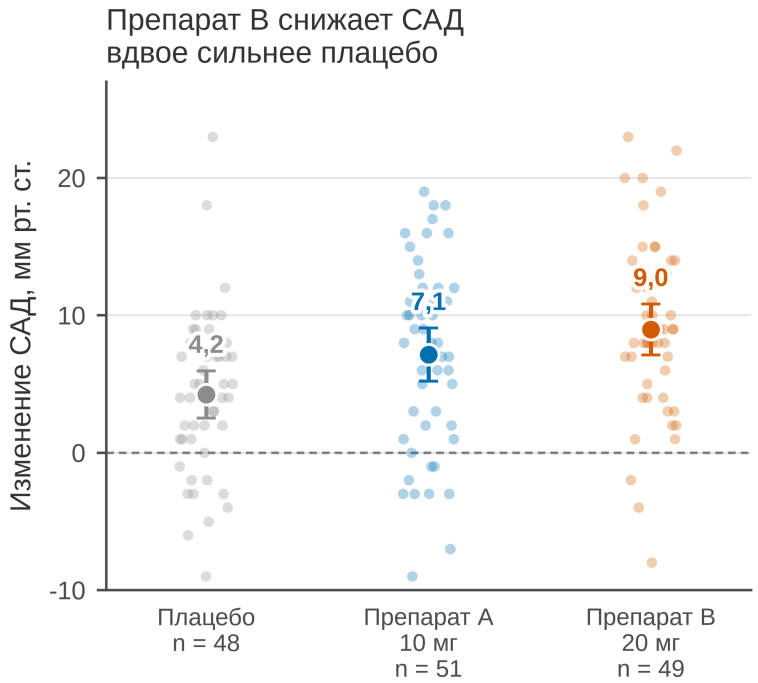


Рисунок 1.2. Правильный график: индивидуальные точки, среднее и 95 % ДИ, нулевая линия, палитра Okabe–Ito, прямые подписи.

Код: `01_principles/scripts/good_plot.py`.

1.4.4 Что улучшено

1. **Точечная оценка вместо длины.** Средние показаны точками, а не столбцами; обрезать ось больше не нужно, ложной разницы не возникает.
2. **Нулевая линия.** Штриховая линия на нуле показывает отсутствие эффекта и помогает оценить клиническую значимость.
3. **Индивидуальные наблюдения.** Полупрозрачные точки показывают разброс, выбросы и то, что у части пациентов САД выросло.
4. **Среднее и 95 % ДИ.** Видна неопределённость оценки; в подписи указано, что это t-интервал.
5. **Семантическая палитра.** Серый — контроль, синий — препарат A, вермилион — препарат B. Цвета различимы при дальтонизме, а подписи групп дублируют цвет.
6. **Прямые подписи и *n*.** Названия групп и число пациентов — прямо под осью; легенда не нужна.
7. **Заголовок-вывод.** «Препарат B снижает САД вдвое сильнее плацебо» сразу задаёт вопрос, на который отвечает график.
8. **Единицы и десятичная запятая.** «Изменение САД, мм рт. ст.» — без догадок; числа в русской типографике.
9. **Иерархия.** Лёгкая сетка только по Y, нет верхней и правой рамок, данные контрастнее осей.
10. **Журнальный размер.** 85 мм в ширину, кегли 7–9,5 pt; рисунок спроектирован в конечном размере, а не уменьшен из «слайда».

1.4.5 Сопоставление

Что было	Почему плохо	Что сделано
ось Y от 3,5	длина столбца искажена, LF ≈ 3,5	точки + нулевая линия
красный/жёлтый/зелёный	ложная семантика, CVD	Okabe–Ito: серый/синий/вермилион
тени, 3D	декор, искажает площадь	плоские маркеры
сетка по X и Y	шум, конкуренция с данными	сетка только по Y, бледная
полная рамка	визуальный шум	только левая и нижняя оси
легенда	дублирует подписи, закрывает данные	прямые подписи под осью
«Результаты исследования»	нет сообщения	заголовок-вывод
нет единиц и *n*	невозможно прочитать	единицы, *n* в подписях
только средние	скрыт разброс	точки + среднее + 95 % ДИ
p < 0,001	нет теста и эффекта	эффект и ДИ; тест — в подписи

Пара честных оговорок:

- График показывает изменение от исходного уровня, но не заменяет статистическую модель. Для сравнения групп с поправкой на исходное САД нужен ANCOVA или смешанная модель; рисунок этого не «исправляет».
- При $n \approx 50$ точек на группу strip plot может выглядеть шумно. Альтернативы — box/violin с точками, half-violin, или среднее с ДИ на фоне распределения. Выбор зависит от сообщения и от того, что важнее: форма распределения или компактность.

1.4.6 Как воспроизвести

```
# из корня репозитория
uv sync
uv run python 01_principles/scripts/make_data.py
uv run python 01_principles/scripts/bad_plot.py
uv run python 01_principles/scripts/good_plot.py
```

Ключевой фрагмент «правильного» кода:

```
from goodgraph import apply_style, figure_size, save_figure

apply_style()
fig, ax = plt.subplots(
    figsize=figure_size("single", ratio=0.90),
    layout="constrained",
)

# ... точки, средние, 95 % ДИ ...

ax.set_ylabel("Изменение САД, мм рт. ст.")
ax.set_title("Препарат В снижает САД\nвдвое сильнее плацебо", loc="left")
save_figure(fig, "01_good_dot_ci")  # images/01_good_dot_ci.svg + .png
```

Доверительный интервал считается через t-распределение, а не «среднее ± 2 SE»:

```
from scipy import stats

se = sd / np.sqrt(n)
half_width = stats.t.ppf(0.975, df=n - 1) * se
ci_low, ci_high = mean - half_width, mean + half_width
```

Для больших n разница с нормальным приближением мала, но t-интервал корректнее при $n \approx 50$ и менее.

1.5 Итоги раздела

Десять тезисов, которые стоит запомнить:

1. График — аргумент, а не украшение; сначала сообщение, потом код.
2. Тип графика определяется задачей и структурой данных.
3. Столбцы — с нуля; точки и линии — могут без нуля, но с объяснением.
4. Логарифм, разрыв оси и две оси Y — только осознанно и с подписями.
5. Показывайте n, разброс и неопределённость; определяйте, что такое усы.
6. Эффект и 95 % ДИ важнее p-значения.
7. Убирайте chartjunk, но не путайте минимализм с потерей информации.
8. Цвет — смысл; палитра должна работать при дальтонизме и в сером.
9. Подпись самодостаточна; рисунок согласован с остальными.
10. Проектируйте в конечном размере и проверяйте в PDF при 100 %.

Дальше — инструменты. В разделе 2 разберём, как это реализуется в Python: matplotlib, seaborn, plotnine, настройка среды через uv и типичные архитектурные решения для воспроизводимых графиков.

Источники к разделу

- Tufte E. R. *The Visual Display of Quantitative Information*. Graphics Press, 1983. — edwardtufte.com
- Cleveland W. S., McGill R. Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods // *JASA*. 1984. Vol. 79, № 387. P. 531–554. — doi:10.1080/01621459.1984.10478080
- Wilke C. O. *Fundamentals of Data Visualization*. O’Reilly, 2019. — clauswilke.com/dataviz
- Cairo A. *The Truthful Art: Data, Charts, and Maps for Communication*. New Riders, 2016. — thefunctionalart.com
- Schwabish J. *Better Data Visualizations: A Guide for Scholars, Researchers, and Wonks*. Columbia University Press, 2021. — policyviz.com
- Okabe M., Ito K. Color Universal Design (CUD): How to Make Figures and Presentations that are Friendly to Colorblind People. 2008. — jfly.uni-koeln.de/color
- ColorBrewer 2.0 — colorbrewer2.org
- Weissgerber T. L. et al. Beyond Bar and Line Graphs: Time for a New Data Presentation Paradigm // *PLoS Biology*. 2015. Vol. 13, № 4. e1002128. — doi:10.1371/journal.pbio.1002128
- ICMJE Recommendations — icmje.org
- EQUATOR Network — equator-network.org
- Nature. Formatting guide — nature.com
- W3C. WCAG 2.1, Understanding SC 1.4.11 Non-text Contrast — w3.org
- ГОСТ 7.32-2017 — docs.cntd.ru
- ГОСТ 2.105-95 — docs.cntd.ru
- ГОСТ 8.417-2002 — docs.cntd.ru

Раздел 2. Python-инструментарий и настройка среды

В разделе 1 мы договорились, каким должен быть график: честным, читаемым, доступным и воспроизводимым. Теперь — чем его рисовать. Задача этого раздела — не выучить все функции matplotlib, а понять, **как устроен стек** и **как получить воспроизводимый результат** из любого места: с ноутбука, из CI, с другого компьютера.

Хорошая новость: инструменты не определяют качество графика. Плохой график можно сделать и в matplotlib, и в ggplot2. Но разные библиотеки по-разному заставляют думать о данных. Matplotlib даёт полный контроль и требует дисциплины. Seaborn берёт на себя статистику и группы. Plotnine заставляет описать график как композицию слоёв. Выбор — не религия, а вопрос задачи.

> Все примеры раздела используют собственный набор `data/02_python_dataset.csv` (HbA1c, 132 пациента). Данные синтетические: раздел про инструменты, а не про новую клиническую гипотезу.

2.1 Карта экосистемы

Инструмент	Роль	Документация	Когда нужен
Python 3.12	язык и рантайм	docs.python.org	всегда
uv	окружения, зависимости, версии Python	docs.astral.sh/uv	всегда
NumPy	массивы, векторизованная математика	numpy.org/doc	расчёты, генерация данных
pandas	таблицы, groupby, CSV	pandas.pydata.org	загрузка и агрегация
SciPy	статистика, распределения, тесты	docs.scipy.org	доверительные интервалы, тесты
matplotlib	рендеринг, Figure/Axes, artists	matplotlib.org	полный контроль, финальная правка
seaborn	статистические графики, long-format	seaborn.pydata.org	EDA, группы, распределения
plotnine	грамматика ggplot2, фасеты, слои	plotnine.org	многопанельные и декларативные графики
adjustText	подписи без наложений	adjusttext.readthedocs.io	точечные подписи
Pillow	растр, композит, конвертация	pillow.readthedocs.io	сборка панелей, проверка PNG

Стек удобно представлять так:

```
numpy / pandas / scipy
        ↓  данные и статистика
matplotlib (Figure, Axes, artists)
        ↓  рендеринг
seaborn / plotnine
        ↓  грамматика, статистика, фасеты
goodgraph (rcParams, палитры, размеры, save)
        ↓
SVG / PNG / PDF
```

Ключевой факт: **seaborn и plotnine в итоге рисуют через matplotlib**. Поэтому понимание `Figure`/`Axes`, `rcParams` и `savefig` полезно даже тем, кто пишет только на plotnine. И наоборот: matplotlib не мешает использовать seaborn для статистики, а потом дорабатывать оси вручную.

2.2 Три подхода к одному графику

2.2.1 matplotlib: императивный контроль

Matplotlib — низкоуровневый движок. Вы создаёте `Figure` (холст), `Axes` (область с осями) и добавляете artists: линии, точки, текст, стрелки, патчи. Каждый элемент — объект с координатами и свойствами.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(3.35, 2.8), layout="constrained")

for group, part in df.groupby("group"):
    ax.scatter(
        part["baseline_hba1c_pct"],
        part["followup_hba1c_pct"],
        s=12,
        alpha=0.5,
        label=group,
    )

ax.set_xlabel("HbA1c до, %")
ax.set_ylabel("HbA1c после, %")
ax.legend(frameon=False)
fig.savefig("scatter.svg")
```

Сильные стороны:

- полный контроль над каждым элементом;
- аннотации, врезки, стрелки, произвольные artists;
- предсказуемое поведение при экспорте;
- огромная экосистема примеров.

Слабые стороны:

- многословно;
- статистика и группировка — вручную;
- легко получить несогласованные рисунки, если каждый скрипт настраивает оси по-своему;
- глобальное состояние `pyplot` (`plt.plot`, `plt.xlim`) может «протечь» из одного рисунка в другой.

Практическое правило: в скриптах используйте **объектный API** (`fig, ax = plt.subplots()`), а не `plt.plot`. Глобальный `pyplot` удобен в интерактивной консоли, но плохо предсказуем в воспроизводимом пайплайне.

2.2.2 seaborn: данные и статистика

Seaborn — надстройка над matplotlib, ориентированная на таблицы данных. Вы передаёте `data=` и имена колонок, а не массивы; `hue`, `style`, `size`, `col`, `row` автоматически кодируют группы и создают фасеты.

```
import seaborn as sns

sns.scatterplot(
    data=df,
    x="baseline_hba1c_pct",
    y="followup_hba1c_pct",
    hue="group",
    style="group",
    ax=ax,
)
```

Сильные стороны:

- статистические преобразования «из коробки»: средние, доверительные интервалы, регрессии, KDE;
- аккуратные умолчания для категориальных групп;
- быстрый EDA и разведочные графики;
- интеграция с matplotlib через `ax=`.

Слабые стороны:

- меньше контроля над отдельными artists;
- темы seaborn (`sns.set_theme`) **перезаписывают** rcParams matplotlib;
- API статистики менялся между версиями (`ci` → `errorbar`), поэтому примеры из старых статей могут не работать;
- легко получить «красиво, но не по журнальному»: крупные шрифты, лишние линии, тени.

Практическое правило: seaborn — для исследования и быстрых статистических сводок; финальную правку удобнее делать в matplotlib.

2.2.3 plotnine: грамматика графики

Plotnine — реализация грамматики графики (Wilkinson; ggplot2) для Python. График описывается декларативно: данные → эстетические отображения → геометрия → шкалы → координаты → фасеты → тема.

```
from plotnine import *

(
    ggplot(df, aes("baseline_hba1c_pct", "followup_hba1c_pct", color="group"))
    + geom_point(alpha=0.5)
    + labs(x="HbA1c до, %", y="HbA1c после, %")
    + theme_bw()
)
```

Сильные стороны:

- декларативность: вы описываете **что**, а не **как**;
- фасеты (`facet_wrap`, `facet_grid`) и согласованные шкалы;
- легко менять геометрию, не переписывая код;
- хорошо ложится на многопанельные статьи и воспроизводимые отчёты.

Слабые стороны:

- требует tidy/long-формата данных;
- меньше контроля над отдельными artists, чем в matplotlib;
- своя система тем, не полностью совпадающая с rcParams;
- часть возможностей ggplot2 отличается или называется иначе;
- ошибки в слоях иногда появляются только на этапе отрисовки.

Практическое правило: plotnine — для структурированных многопанельных графиков и случаев, где важна декларативная воспроизводимость.

2.2.4 Сравнение

Критерий	**matplotlib**	**seaborn**	**plotnine**
Уровень	низкий	средний	высокий
Стиль	императивный	дата-ориентированный	декларативный
Контроль над artists	максимальный	средний	средний/низкий
Статистика «из коробки»	нет	да	частично
Фасеты	вручную	`col`/`row`	`facet_wrap`/`facet_grid`
Кривая обучения	средняя	низкая	средняя
Лучше всего для	финальной правки, аннотаций	EDA, групп, распределений	структуры, multi-panel

Это не «или». Типичный пайплайн: seaborn или plotnine для черновика, matplotlib для финальной правки, `goodgraph` — для единого стиля.

2.3 Один набор данных — три библиотеки

Чтобы сравнение не осталось теоретическим, один и тот же набор (HbA1c до и после 12 недель, 132 пациента) построен трижды. Код — `02_python/scripts/compare_libraries.py`.

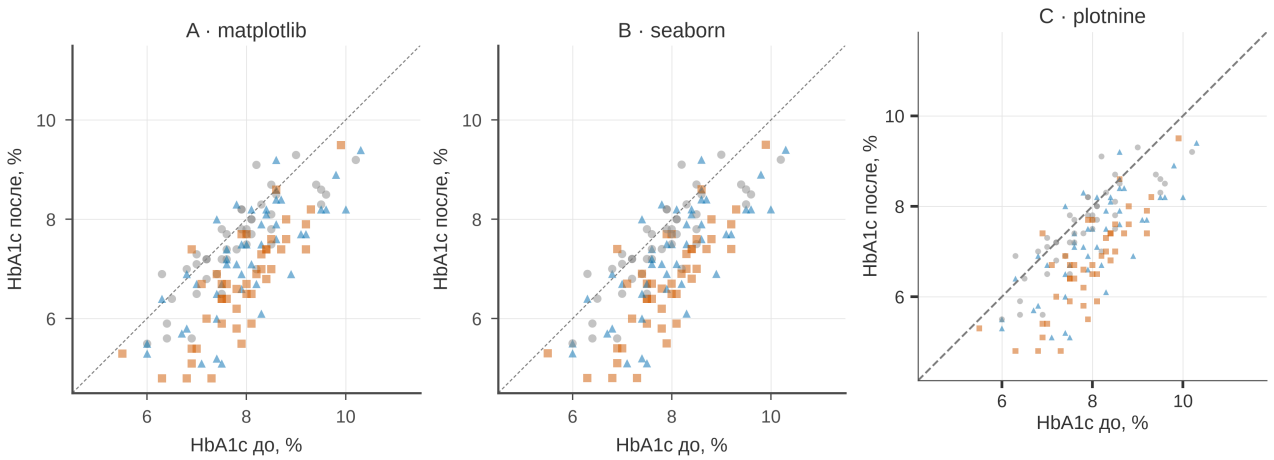


Рисунок 2.1. Один и тот же набор данных в трёх библиотеках: A — matplotlib, B — seaborn, C — plotnine. Серые круги — плацебо, синие треугольники — препарат A 10 мг, вермилионовые квадраты — препарат B 20 мг; штриховая линия — равенство HbA1c до и после. Данные — `data/02_python_dataset.csv`.

Что здесь важно:

- **Общие пределы и деления.** Все панели используют `xlim/ylim = 4,5–11,5` и деления 6/8/10. Иначе сравнение было бы нечестным.
- **Общая палитра и формы.** Цвет и форма маркера заданы один раз и повторены во всех библиотеках. Это принцип согласованности из раздела 1.
- **Разный объём кода.** Matplotlib требует явного цикла по группам; seaborn — одного вызова с `hue`/`style`; plotnine — описания слоёв и темы. Но результат один и тот же, потому что стиль вынесен наружу.
- **Размер панели 54 × 62 мм.** Три панели по 54 мм с промежутками дают ширину около 170 мм. Это пример проектирования в конечном размере, а не «нарисуем большое и уменьшим».

Ключевые фрагменты:

```
# matplotlib: цикл по группам
for group in GROUP_ORDER:
    part = df.loc[df["group"] == group]
    ax.scatter(part["baseline_hba1c_pct"], part["followup_hba1c_pct"],
               marker=GROUP_MARKERS[group], color=GROUP_COLORS[group])

# seaborn: одна функция, группы через hue/style
sns.scatterplot(data=df, x="baseline_hba1c_pct", y="followup_hba1c_pct",
                hue="group", style="group", ax=ax)

# plotnine: слои через «+»
(
    ggplot(df, aes("baseline_hba1c_pct", "followup_hba1c_pct",
                   color="group", shape="group"))
    + geom_point(alpha=0.5)
    + geom_abline(slope=1, intercept=0, linetype="dashed")
    + coord_fixed(ratio=1, xlim=(80, 215), ylim=(80, 215))
    + theme_bw()
)
```

2.4 uv: окружение и воспроизводимость

2.4.1 Что такое uv

`uv` — менеджер пакетов и окружений от Astral, написанный на Rust. Он заменяет связку `pip` + `venv` + `pip-tools` + `pyenv` и умеет:

- создавать виртуальные окружения;
- разрешать и фиксировать зависимости (`uv.lock`);
- скачивать и переключать версии Python;

- запускать скрипты и инструменты (`uv run`, `uvx`);
- работать в CI и на чужих машинах.

Документация: docs.astral.sh/uv. В этом репозитории используется uv 0.12 и Python 3.12; точные версии пакетов зафиксированы в `uv.lock`.

2.4.2 Установка

```
# Linux / macOS
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows PowerShell
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# Если уже есть pipx или Homebrew
pipx install uv
brew install uv

uv --version
```

Если нужной версии Python нет:

```
uv python install 3.12
uv python list
```

`uv` скачает интерпретатор в свой каталог и не тронет системный Python.

2.4.3 Новый проект

```
mkdir my-figure && cd my-figure
uv init --python 3.12
uv add matplotlib seaborn plotnine pandas numpy scipy
uv add --dev ruff pytest
uv run python -c "import matplotlib; print(matplotlib.__version__)"
```

Что появилось:

- `pyproject.toml` — метаданные проекта и зависимости;
- `.python-version` — закреплённая версия Python;
- `uv.lock` — точное дерево зависимостей с хешами;
- `.venv/` — виртуальное окружение (не коммитится).

`uv add` одновременно обновляет `pyproject.toml`, `uv.lock` и окружение. `uv sync` приводит окружение в соответствие с lock-файлом.

2.4.4 `pyproject.toml` этого репозитория

Сокращённый вид:

```
[project]
name = "goodgraph"
version = "0.1.0"
requires-python = ">=3.12,<3.13"
dependencies = [
    "numpy>=2.0",
    "pandas>=2.2",
    "matplotlib>=3.9",
    "seaborn>=0.13",
    "plotnine>=0.13",
    "scipy>=1.13",
]

[dependency-groups]
dev = ["ruff>=0.6", "pytest>=8.0"]

[tool.uv]
package = true
```

Полная версия с `[build-system]` и `[tool.hatch.build.targets.wheel]` — в `pyproject.toml`.

Что здесь важно:

- `requires-python` задаёт **диапазон**, а `.python-version` — конкретную версию для разработки;
- `uv.lock` фиксирует точные версии; его нужно коммитить;
- `[tool.uv] package = true` и `src/`-layout означают, что пакет `goodgraph` устанавливается в окружение редактируемым (`pip install -e`);
- `[dependency-groups] dev` — инструменты разработки, которые не нужны читателю в рантайме.

2.4.5 Повседневные команды

Команда	Что делает
`uv sync`	создаёт/обновляет `.venv` по `uv.lock`
`uv sync --frozen`	не меняет lock, падает при расхождении — для CI
`uv add pandas`	добавляет зависимость и обновляет lock
`uv add --dev ruff`	добавляет в dev-группу
`uv remove pandas`	удаляет зависимость
`uv lock`	пересчитывает lock
`uv lock --upgrade`	обновляет версии в пределах ограничений
`uv run python script.py`	запускает в окружении проекта
`uv run --with rich script.py`	добавляет пакет только на запуск

Команда	Что делает
`uv python pin 3.12`	фиксирует версию Python
`uv tree`	показывает дерево зависимостей
`uv pip list`	список установленного
`uvx ruff check .`	запускает инструмент без установки в проект

`uv run` не требует активации окружения. Если привычнее `source .venv/bin/activate`, это тоже работает; но в скриптах и CI `uv run` надёжнее — он гарантирует нужное окружение.

2.4.6 Одноразовые скрипты и PEP 723

Если скрипт не входит в проект, зависимости можно объявить прямо в нём:

```
# /// script
# requires-python = ">=3.12"
# dependencies = ["matplotlib", "pandas"]
# ///

import matplotlib.pyplot as plt
import pandas as pd
```

Запуск:

```
uv run plot.py
```

`uv` создаст временное окружение с указанными пакетами и выполнит скрипт. Это удобно для примеров в статье или для утилиты, которую не хочется тащить в основной `pyproject.toml`. Спецификация — PEP 723.

2.4.7 Воспроизводимость и кеш

Правила:

- коммитить `pyproject.toml`, `uv.lock`, `.python-version`;
- не коммитить `.venv/`;
- в CI использовать `uv sync --frozen` и `uv run ...`;
- обновлять зависимости осознанно: `uv lock --upgrade`, затем прогнать тесты и пересобрать рисунки;
- фиксировать системные зависимости отдельно: шрифты, Cairo, LaTeX, Graphviz — uv их не устанавливает.

В песочнице или CI с read-only домашним каталогом задайте локальные пути:

```
export UV_CACHE_DIR="$PWD/.uv-cache"
export UV_PYTHON_INSTALL_DIR="$PWD/.uv-python"
export MPLCONFIGDIR="$PWD/.mplconfig"
```

В этом репозитории такие каталоги уже добавлены в `.gitignore`.

2.5 Анатомия воспроизводимого графика

2.5.1 Четыре слоя

```
data/02_python_dataset.csv
      ↓  make_data.py: seed, генерация, CSV
      ↓  plot.py: pandas/scipy, агрегация
      ↓  goodgraph.style: rcParams, палитра, размер
      ↓  matplotlib / seaborn / plotnine
      ↓  save_figure: SVG + PNG
images/01_good_dot_ci.svg, .png
```

Принципы:

1. **Данные не знают о графике.** `make_data.py` пишет CSV и не импортирует matplotlib.
2. **График не генерирует данные.** `plot.py` читает CSV; если данных нет — падает с понятной ошибкой.

3. **Стиль — отдельный слой.** Один `apply_style()` и одна палитра на всю методичку.
4. **Экспорт — отдельная функция.** `save_figure()` сохраняет SVG и PNG, закрывает фигуру и печатает пути.

2.5.2 Каркас скрипта

```python
from __future__ import annotations

import matplotlib

matplotlib.use("Agg")  # без дисплея

import matplotlib.pyplot as plt

from goodgraph import apply_style, figure_size, save_figure
from goodgraph.paths import DATA_DIR

apply_style()
fig, ax = plt.subplots(
    figsize=figure_size("single", ratio=0.8),
    layout="constrained",
)
# ... данные и слои ...
save_figure(fig, "02_example")
```

`figure_size("single")` возвращает размер в дюймах для 85 мм; `save_figure` сохраняет `images/02_example.svg` и `images/02_example.png` с dpi 600 и закрывает фигуру. Если нужен точный физический размер, не включайте `tight=True`: обрезка полей меняет ширину.

2.5.3 Что задаёт `apply_style`

Параметр	Значение	Зачем
`font.family`	Liberation Sans, Arial, DejaVu Sans	кириллица, метрики Arial
`font.size`	9 pt (paper)	журнальный кегль
`axes.spines.top/right`	False	меньше визуального шума
`axes.grid`	True, только Y	помогает читать значения
`grid.color`	`#E6E6E6`	сетка не конкурирует с данными
`axes.prop_cycle`	Okabe–Ito	доступная палитра по умолчанию
`svg.fonttype`	`path`	текст не «поплывёт» в SVG
`pdf.fonttype`	42	TrueType в PDF, текст редактируем
`savefig.dpi`	600	достаточный растр
`axes.unicode_minus`	False	совместимость со шрифтами

2.5.4 Порядок применения стилей

Порядок важен, потому что настройки перекрывают друг друга:

1. значения по умолчанию matplotlib;
2. `plt.style.use("...")`, если он используется;
3. `sns.set_theme(...)` — **сбрасывает** многие rcParams;
4. `apply_style(...)` — должен идти последним;
5. точечные переопределения в конкретном графике.

Если вызвать `sns.set_theme()` после `apply_style()`, шрифты, рамки и сетка могут вернуться к умолчаниям seaborn. Если нужна тема seaborn — передайте её параметры в `apply_style` или настройте rcParams после `set_theme`.

Plotnine живёт в своей системе тем. `theme(text=element_text(...))` не читает все rcParams matplotlib; задавайте шрифт и размеры явно.

2.6 Разбор `compare_libraries.py`

Скрипт `02_python/scripts/compare_libraries.py` устроен так:

1. **Общие константы.** `WIDTH_MM = 54`, `HEIGHT_MM = 62`, `LIM_MIN = 80`, `LIM_MAX = 215`, `TICKS = [80, 120, 160, 200]`. Одинаковые пределы — обязательное условие честного сравнения.
2. **Палитра и формы.** `GROUP_COLORS` и `GROUP_MARKERS` заданы один раз; цвет дублируется формой, чтобы панели читались в сером.
3. **`load_data()`.** Читает `data/02_python_dataset.csv` и проверяет, что все группы на месте.
4. **`style_axes()`.** Общая функция для matplotlib и seaborn: пределы, деления, линия равенства, подписи, сетка, аспект.
5. **Три функции построения.** `plot_matplotlib`, `plot_seaborn`, `plot_plotnine` — каждая самодостаточна и сохраняет свою пару SVG/PNG.
6. **`main()`.** Последовательно вызывает три функции.

Почему это важно: если каждая панель настраивает пределы сама, сравнение превращается в сравнение разных графиков. Общая функция `style_axes` фиксирует контракт.

2.7 Типичные ошибки и как их исправить

2.7.1 `plt.show()` в скрипте

```
# плохо: скрипт зависит от дисплея
plt.show()

# хорошо: Agg + сохранение
matplotlib.use("Agg")
fig.savefig("figure.png", dpi=600)
```

`show()` полезен в интерактивной консоли; в скрипте, CI и на сервере он либо блокирует выполнение, либо падает без дисплея.

2.7.2 Глобальное состояние `pyplot`

```
# плохо: неясно, какая фигура сохраняется
plt.plot(x, y)
plt.title("...")
plt.savefig("a.png")
plt.plot(x2, y2)
plt.savefig("b.png")  # на самом деле обе линии на одной фигуре

# хорошо: явные объекты
fig, ax = plt.subplots()
ax.plot(x, y)
fig.savefig("a.png")
plt.close(fig)
```

2.7.3 `sns.set_theme()` после `apply_style()`

```
# плохо: тема seaborn перезапишет шрифты и рамки
apply_style()
sns.set_theme(style="whitegrid")

# хорошо: сначала тема, потом свой стиль
sns.set_theme(style="whitegrid")
apply_style()
```

2.7.4 `tight_layout` и `constrained_layout` вместе

```
# плохо: конфликтующие движки компоновки
fig, ax = plt.subplots(layout="constrained")
fig.tight_layout()

# хорошо: один движок
fig, ax = plt.subplots(layout="constrained")
```

2.7.5 `bbox_inches="tight"` и точная ширина

```
# плохо для журнальной ширины: итоговый размер может отличаться от 85 мм
fig.savefig("figure.svg", bbox_inches="tight")

# хорошо: размер задан figsize, поля — полями
fig.savefig("figure.svg")
```

`tight` удобен для слайдов и препринтов, но не для точного соответствия колонке журнала.

2.7.6 Шрифт не найден

```
from matplotlib import font_manager

print(font_manager.findfont("Liberation Sans"))
```

Если шрифт не найден, matplotlib молча подставит DejaVu Sans и предупредит. Проверяйте `findfont` и наличие кириллицы. В Linux установите `fonts-liberation`, в Windows используйте Arial.

2.7.7 SVG с текстом или с кривыми

- `svg.fonttype = "path"` — текст превращается в кривые: вид стабилен, но текст нельзя редактировать.
- `svg.fonttype = "none"` — текст остаётся текстом: удобно для правки, но зависит от шрифтов у читателя.

Для журнала обычно выбирают `path`; для внутренней правки — `none`. В этом проекте по умолчанию `path`.

2.7.8 Единицы plotnine

```
# плохо: дюймы и миллиметры перепутаны
plot.save("figure.png", width=85, height=60)

# хорошо: перевести миллиметры в дюймы
plot.save("figure.png", width=85 / 25.4, height=60 / 25.4, dpi=600)
```

`plotnine.save` принимает ширину и высоту в дюймах, как и matplotlib.

2.7.9 Данные не в tidy-формате

Plotnine ожидает long-формат: одна строка — одно наблюдение, колонки — переменные. Широкая таблица `group, before, after` подойдёт для matplotlib, но для plotnine её нужно привести к `group, time, value` через `melt`.

2.7.10 DPI и цветовой профиль

- DPI влияет только на растр; для SVG/PDF он не важен.
- RGB → CMYK лучше конвертировать в конце, после утверждения макета; проверьте, что цвета не «провалились» в печати.
- Прозрачный фон (`transparent=True`) почти никогда не нужен для журнала: белый фон предсказуемее.

2.8 Чек-лист раздела 2

- ☐ `uv sync --frozen` работает на чистой машине.
- ☐ `pyproject.toml`, `uv.lock`, `.python-version` закоммичены.
- ☐ `.venv/` не закоммичен.
- ☐ Скрипт использует `matplotlib.use("Agg")`.
- ☐ Данные и построение разделены: `make_data.py` ≠ `plot.py`.
- ☐ `apply_style()` вызывается один раз и последним.
- ☐ Нет `sns.set_theme()` после `apply_style()`.
- ☐ Пределы осей и палитра заданы явно, а не «по умолчанию».
- ☐ Сохраняются SVG и PNG; размер — в миллиметрах, не «как получилось».
- ☐ Шрифт найден, кириллица и символы (μ, ±, ≥) отображаются.
- ☐ `uv run python ...` работает из корня репозитория.
- ☐ `uv run ruff check .` проходит.
- ☐ Повторный запуск даёт те же данные и рисунки (фиксированный seed).

2.9 Итоги

- **matplotlib** — фундамент и полный контроль; используйте объектный API.
- **seaborn** — быстрый доступ к статистике и группам; не заменяет matplotlib, а дополняет его.
- **plotnine** — грамматика графики и фасеты; требует tidy-данных и отдельной темы.
- **uv** — воспроизводимость: версия Python, lock-файл, `uv run`, `uv sync --frozen`.
- **goodgraph** — единый стиль и экспорт; это контракт между разделами.

Дальше — раздел 3: требования к цвету, шрифтам, размерам и форматам. Там мы разберём палитры подробнее, научимся проверять рисунок на дальтонизм и печать и соберём таблицу «параметр → значение → источник».

Источники к разделу

- Matplotlib documentation — matplotlib.org
- Matplotlib: Quick start guide — matplotlib.org
- Matplotlib: Artists — matplotlib.org
- Seaborn documentation — seaborn.pydata.org
- Seaborn tutorial — seaborn.pydata.org/tutorial
- Plotnine documentation — plotnine.org
- Plotnine reference — plotnine.org/reference
- uv documentation — docs.astral.sh/uv
- uv: Working on projects — docs.astral.sh/uv/guides/projects
- uv: Running scripts — docs.astral.sh/uv/guides/scripts
- PEP 723: Inline script metadata — peps.python.org/pep-0723
- Wilkinson L. *The Grammar of Graphics*. 2nd ed. Springer, 2005. — doi:10.1007/0-387-28695-0
- ggplot2 documentation — ggplot2.tidyverse.org
- Rougier N. P., Droettboom M., Bourne P. E. Ten Simple Rules for Better Figures // *PLoS Computational Biology*. 2014. Vol. 10, № 9. e1003833. — doi:10.1371/journal.pcbi.1003833
- pandas documentation — pandas.pydata.org
- NumPy documentation — numpy.org/doc
- SciPy documentation — docs.scipy.org
- adjustText documentation — adjusttext.readthedocs.io
- Pillow documentation — pillow.readthedocs.io

Раздел 3. Требования к научной графике: цвет, шрифты, размеры и форматы

В разделе 1 мы сформулировали принципы, в разделе 2 — выбрали инструменты. Теперь наступает самая практичная часть: **конкретные требования**, которые можно проверить до отправки статьи. Не «сделайте красиво», а «ширина 85 мм, кегль не меньше 7 pt, контраст не ниже 3:1, шрифт встроен».

Требования удобно разделить на три уровня:

1. **Смысл** — что именно кодирует цвет, размер, положение.
2. **Доступность** — увидят ли это при дальтонизме, в сером, на печати.
3. **Производство** — дойдёт ли файл до редакции без потерь.

Гайд журнала всегда важнее любых рекомендаций ниже. Если журнал требует RGB, 300 dpi и TIFF — делайте RGB, 300 dpi и TIFF, даже если раздел советует вектор. Но если гайд молчит, значения по умолчанию из этого раздела помогут не изобретать велосипед.

3.1 Цвет: язык, а не украшение

3.1.1 Что кодирует цвет

Цвет в научном графике выполняет одну из четырёх ролей:

- **категория** — группа, препарат, регион, тип события;
- **величина** — концентрация, время, доза, интенсивность;
- **отклонение** — разница с нулём, нормой или контролем;
- **акцент** — главный вывод на фоне серого контекста.

Цвет — мощный, но неточный канал. Точные значения читаются по положению и длине, а цвет хорош для **группировки** и **поиска**. Поэтому не стоит кодировать цветом то, что читатель должен сравнивать количественно: для этого есть оси, точки и подписи.

Три практических следствия:

- цвет должен иметь смысл: если убрать цвет, график не должен потерять информацию (или должен потерять только второстепенную);
- одно и то же соответствие «цвет → группа» сохраняется во всех рисунках;
- цвет не единственный носитель смысла: форма, тип линии, подпись дублируют его.

3.1.2 Три типа палитр

Тип	Вопрос	Примеры	Правило
Качественная	«Какая это категория?»	Okabe–Ito, Paul Tol bright/muted, ColorBrewer Set2	6–8 цветов, без порядка, без радуги
Последовательная	«Насколько велика величина?»	viridis, cividis, Blues, Greens	монотонная светлота, равномерные шаги
Дивергентная	«Насколько отличается от нуля?»	RdBu_r, PuOr, BrBG	два полюса, нейтральная середина в смысловом нуле

Типичная ошибка — использовать последовательную шкалу для категорий («группа 1 — светло-синяя, группа 2 — тёмно-синяя»): читатель начинает искать порядок и величину там, где их нет. Обратная ошибка — раскрасить упорядоченную величину качественными цветами: пропадает направление изменения.

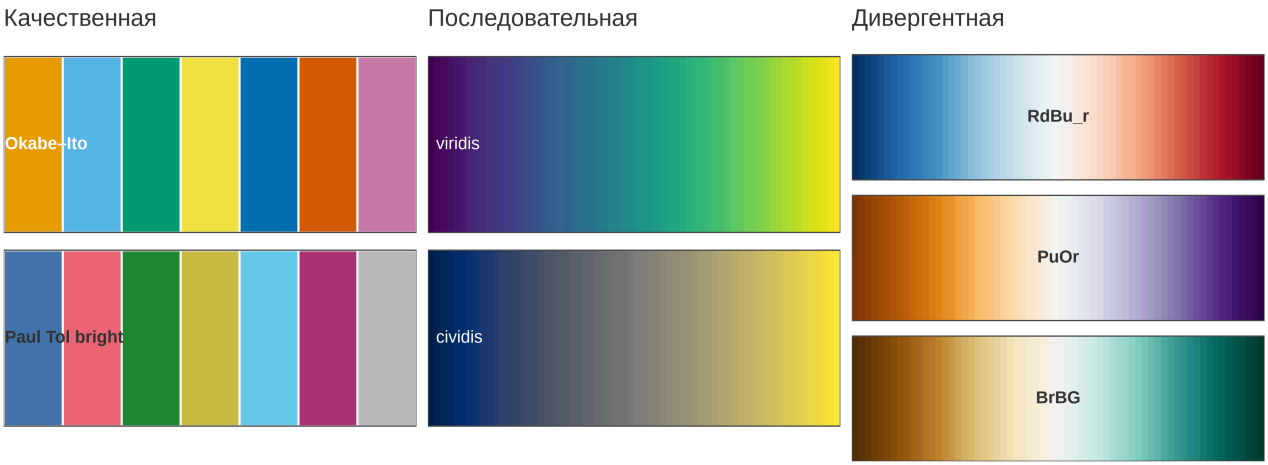


Рисунок 3.1. Три типа палитр: качественная (Okabe–Ito, Paul Tol bright), последовательная (viridis, cividis) и дивергентная (RdBu_r, PuOr, BrBG). Качественная отвечает на вопрос «кто это», последовательная — «сколько», дивергентная — «насколько отличается от нуля».

3.1.3 Качественные палитры

Базовый набор — **Okabe–Ito**. Он различим при протанопии и дейтеранопии и хорошо работает в полиграфии.

Цвет	HEX	Роль
оранжевый	`#E69F00`	категория 1
небесно-голубой	`#56B4E9`	категория 2
сине-зелёный	`#009E73`	категория 3
жёлтый	`#F0E442`	категория 4, только с тёмным контуром
синий	`#0072B2`	основной результат
вермилион	`#D55E00`	акцент/второй результат
красновато-фиолетовый	`#CC79A7`	категория 5
чёрный	`#000000`	текст, оси, а не серии

Дополнительные варианты:

- **Paul Tol bright/muted** — спокойные наборы для больших заливок и слайдов;
- **ColorBrewer** — проверенные качественные, последовательные и дивергентные схемы, в том числе для печати;
- **табличные** палитры (например, `tab10`) удобны, но не оптимизированы под дальтонизм; не полагайтесь на них в финальной версии.

Правила:

- не больше 6–8 категориальных цветов; больше — объединяйте редкие категории в «прочее» или делайте small multiples;
- серый — для контроля, фона и «прочего»; он не должен конкурировать с данными;
- один акцентный цвет на главный вывод; остальное — сдержанное;
- не используйте красный и зелёный как единственную пару «плохо/хорошо»;
- проверяйте палитру в сером и через симулятор дальтонизма.

3.1.4 Последовательные и дивергентные шкалы

Последовательная шкала должна:

- монотонно менять светлоту — тогда она читается в сером;
- быть равномерной по восприятию — без ложных границ;
- подходить для дальтоников;
- иметь подписанную цветовую шкалу с единицами.

`viridis` и `cividis` — хорошие значения по умолчанию. `cividis` специально спроектирован для дальтонизма. `Blues` и `Greens` хороши для печати, но следите за контрастом крайних значений.

```python
import matplotlib as mpl

cmap = mpl.colormaps["viridis"]
# Для дискретных значений:
cmap = mpl.colormaps["viridis"].resampled(5)
```

Почему не `jet` и не «радуга»:

- светлота меняется немонотонно, возникают ложные границы;
- разные шаги данных выглядят как разные по величине;
- жёлтый и голубой на белом фоне теряют контраст;
- шкала плохо читается при дальтонизме и в сером.

Дивергентная шкала нужна, когда есть **смысловой ноль**: изменение, разница, отклонение от нормы. Правила:

- центр шкалы — ровно ноль (или другая содержательная точка);
- пределы симметричны: −3…+3, а не −1…+5;
- два полюса разного цвета, середина — нейтральная;
- не используйте дивергентную шкалу для данных без нуля.

Белая середина `RdBu_r` на белом фоне может «исчезнуть». Если это мешает, добавьте тонкую контурную линию или выберите середину с лёгким оттенком.

3.1.5 Доступность: дальтонизм и серый

Около 8 % мужчин европейского происхождения имеют нарушения цветового зрения; среди женщин доля ниже, но не нулевая. Наиболее часты протанопия и дейтеранопия (красный/зелёный), реже — тританопия (синий/жёлтый).

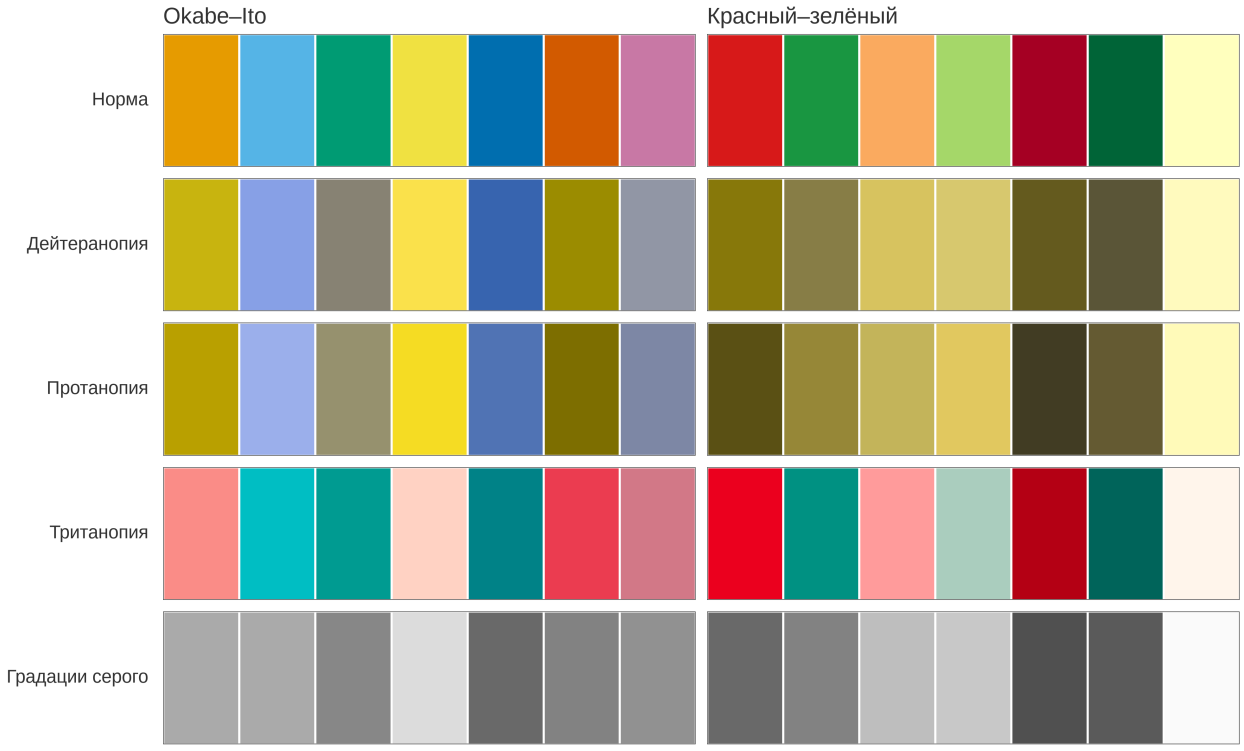


Рисунок 3.2. Симуляция дальтонизма (Machado et al., 2009; severity 100 %). Слева — Okabe–Ito, справа — палитра «красный–зелёный». В нижних рядах видно, что красные и зелёные пары сливаются, а Okabe–Ito сохраняет больше различий. В сером ряду цвет исчезает полностью — форму и подписи нужно дублировать.

Что делать:

- **дублируйте цвет формой, типом линии, штриховкой или прямой подписью**;
- не кодируйте противоположные категории только красным и зелёным;
- проверяйте рисунок в градациях серого;
- проверяйте контраст: текст — не ниже 4,5:1, графические объекты — не ниже 3:1 (WCAG 2.1);

- используйте симуляторы (Coblis, Color Oracle, Photoshop, матрицы Machado) и, в идеале, покажите рисунок человеку с дальтонизмом.

Серый — не «отсутствие цвета», а полноценный канал. В сером графике работают положение, форма, толщина линии и подписи. Если после обесцвечивания серии неразличимы — добавьте форму или подпись.

3.1.6 Цвет в печати

Экран и бумага — разные устройства.

- **sRGB** — для онлайн-публикации и большинства журнальных PDF.
- **CMYK** — для офсетной печати; охват уже, насыщенные зелёные и синие могут «провалиться».
- Конвертируйте в CMYK **в конце**, после утверждения макета, и проверяйте цветопробу.
- Не полагайтесь на светлые оттенки для тонких линий: на бумаге они исчезают.
- Мелкий текст — 100 % K, а не составной «богатый чёрный»: иначе возможны проблемы с приводкой.
- Не используйте прозрачность как единственный способ показать неопределённость: при печати она может стать грязной.

3.1.7 Чек-лист цвета

- ☐ Цвет кодирует смысл, а не украшает.
- ☐ Тип палитры соответствует задаче: категория / величина / отклонение.
- ☐ Категориальных цветов не больше 6–8.
- ☐ Нет радуги и `jet`; последовательная шкала монотонна по светлоте.
- ☐ Дивергентная шкала центрирована в смысловом нуле и симметрична.
- ☐ Красный и зелёный не единственные носители смысла.
- ☐ Рисунок читается в градациях серого.
- ☐ Контраст текста ≥ 4,5:1, графики ≥ 3:1.
- ☐ Цвета согласованы между рисунками.
- ☐ Проверена цветопроба или CMYK-версия, если журнал печатный.

3.2 Шрифты

3.2.1 Гарнитура

Для рисунков почти всегда лучше **sans-serif**: Arial, Helvetica, Liberation Sans, DejaVu Sans. Они плотнее и лучше читаются в мелком кегле, чем засечные гарнитуры. Основной текст статьи может быть serif — это нормально; важно, чтобы рисунок не спорил с текстом.

Правила:

- одна гарнитура на рисунок, максимум две на статью;
- не использовать декоративные и рукописные шрифты;
- проверять метрическую совместимость: Liberation Sans близок к Arial, поэтому вёрстка не «поедет» при замене;
- встраивать шрифт в PDF или переводить текст в кривые в SVG.

3.2.2 Кегли

Кегль задаётся **в конечном физическом размере**. Если рисунок будет 85 мм, не рисуйте его в 16 дюймов и не уменьшайте в Word: 9 pt превратятся в лотерею.

Элемент	**Ориентир**
Заголовок внутри рисунка	9–10 pt
Подписи осей	8–9 pt
Деления и подписи делений	7–8 pt
Легенда	7–8 pt
Метки панелей A/B/C	9–10 pt, bold
Подпись к рисунку (caption)	8–9 pt, в тексте

Практический минимум — **7 pt**. Некоторые журналы допускают 5–7 pt, но это уже риск: в печати и на плохом экране такой текст расплывается. 6 pt — ниже практического минимума; используйте только как крайнюю меру.



Рисунок 3.3. Один и тот же текст при 6–12 pt в конечном размере 85 мм. В PDF рисунок показан 1:1; в HTML он может масштабироваться, поэтому ориентируйтесь на подписи кеглей, а не на размер картинки на экране.

3.2.3 Начертания, регистр, математика

- Regular — по умолчанию; bold — только для акцента.
- Не писать подписи КАПСОМ: это замедляет чтение.
- Не подчёркивать и не добавлять тени/обводки ради «красоты».
- Математические переменные — курсивом: *p*, *n*, *M*, *SD*.
- Единицы — прямым: мм рт. ст., мг, %, °C.
- Не смешивать курсив и прямой шрифт в одном обозначении без причины.
- Десятичный разделитель — по языку публикации: запятая в русском тексте, точка в англоязычном; главное — единообразие.

3.2.4 Кириллица и специальные символы

Проверяйте не только буквы, но и символы:

- μ, ±, ≥, ≤, →, ≈, ×, Δ, °C;
- минус и дефис — разные символы; в осях допустим обычный минус, но он не должен превращаться в дефис;
- неразрывный пробел между числом и единицей: 8,9 мм, 10 мг, 5 %.

Если шрифт не найден, matplotlib молча подставит DejaVu Sans и предупредит. Проверяйте:

```
from matplotlib import font_manager

print(font_manager.findfont("Liberation Sans"))
```

Для проверки глифов в шрифте можно использовать `fontTools`:

```
from fontTools.ttLib import TTFont

font = TTFont("path/to/font.ttf")
cmap = font.getBestCmap()
print(all(ord(ch) in cmap for ch in "μ±≥≤→≈×Δ°C"))
```

3.2.5 Композиция текста

- Подписи осей — параллельно осям, не под углом 45° без необходимости.
- Единицы — в подписи оси: «САД, мм рт. ст.», а не «САД (мм рт. ст.)» и не отдельной строкой в легенде.
- Деления не должны налезать друг на друга; при необходимости уменьшите их число, а не кегль.
- Легенда — тем же кеглем, что и деления; без рамки, если позволяют оси.
- Прямые подписи лучше легенды, если серий немного.
- Не располагайте подпись поверх данных без белой подложки или ореола.

3.2.6 Чек-лист шрифтов

- ☐ Одна гарнитура, sans-serif для рисунка.
- ☐ Кегли 7–10 pt в конечном размере.
- ☐ Нет КАПСА и декоративных эффектов.
- ☐ Математика курсивом, единицы прямым.
- ☐ Кириллица и символы (μ, ±, ≥, →, °C) отображаются.
- ☐ Десятичный разделитель единообразен.
- ☐ Подписи не налезают друг на друга и не обрезаны.
- ☐ Шрифт встроен в PDF или переведён в кривые в SVG.

3.3 Размеры и композиция

3.3.1 Ширина колонки

Ширина	Типичное применение
85 мм	одна колонка
114 мм	полторы колонки
170 мм	две колонки
180 мм	максимальная ширина полосы

Nature ориентирует на 89 и 183 мм; другие журналы — на 85–90 и 170–183 мм. Проверяйте гайд. Проектируйте сразу в конечной ширине: `figsize` в matplotlib задаётся в дюймах, поэтому 85 мм = 3,35 дюйма, 170 мм = 6,69.

3.3.2 Поля и отступы

- Внешние поля — 3–5 мм, чтобы подписи не обрезались.
- Внутренние отступы между панелями — 2–4 мм.
- Не используйте `bbox_inches="tight"` для журнального рисунка: он меняет физический размер.
- `layout="constrained"` предпочтительнее ручного `subplots_adjust`, если нет особых требований.
- Проверяйте, что подписи осей и легенда не выходят за холст.

3.3.3 Пропорции

- Временной ряд — широкий: 16:9 или 3:2, чтобы тренд читался.
- Scatter — ближе к квадрату, особенно если сравниваете наклоны.
- Столбцы — не слишком широкие: расстояние между ними важнее толщины.
- Не растягивайте рисунок непропорционально: это меняет восприятие наклонов и площадей.
- Для временных рядов полезно «banking to 45°»: подобрать аспект так, чтобы типичный наклон был около 45°.

3.3.4 Панели и выравнивание

- Метки A, B, C — слева сверху, одним кеглем и в одном месте.
- Одинаковые пределы осей, если панели сравниваются.
- Общая легенда — для всех панелей, а не отдельная в каждой.
- Выравнивайте области построения, а не только внешние рамки.
- Не дублируйте подписи осей на каждой панели, если они общие.

3.3.5 Чек-лист размеров

- ☐ Ширина соответствует колонке журнала.
- ☐ Рисунок спроектирован в конечном размере, а не уменьшен.
- ☐ Поля 3–5 мм, ничего не обрезано.
- ☐ Панели выровнены, метки A/B/C на месте.
- ☐ Пропорции не искажены.
- ☐ `bbox_inches="tight"` не используется для точной ширины.

3.4 Форматы и разрешение

3.4.1 Вектор или растр

Формат	Тип	Когда использовать	Ограничения
PDF	вектор	графики, схемы, финальная отправка	проверьте встраивание шрифтов
SVG	вектор	веб, правка в Inkscape/Illustrator	текст может стать кривыми

Формат	Тип	Когда использовать	Ограничения
EPS	вектор	старые редакции, LaTeX	устаревает, проверяйте шрифты
AI	вектор	правка в Illustrator	не все журналы принимают
TIFF	растр	фото, микроскопия	большой файл; LZW без потерь
PNG	растр	веб, скриншоты	не для печати больших фото
JPEG	растр	только фото	артефакты на линиях и тексте

Правило: **линии, текст и графики — вектор; фото — растр**. Не вставляйте скриншот графика в статью: это растр с потерями и без масштабирования.

3.4.2 Разрешение

- Фото и микроскопия — 300 dpi в конечном размере.
- Штриховая графика, если журнал требует растр, — 600–1200 dpi.
- Эффективное разрешение = пиксели / дюймы конечного размера. Например, 2000 px при ширине 3,35″ дают ≈ 597 dpi.
- Не увеличивайте растр интерполяцией: это не добавляет деталей.
- Проверяйте, что после масштабирования в Word разрешение не упало.

3.4.3 Цветовые профили

- sRGB IEC61966-2.1 — безопасный профиль для онлайн-публикации.
- Adobe RGB — шире, но не все редакции его принимают.
- CMYK — для печати; конвертируйте в конце и проверяйте цветопробу.
- Встраивайте профиль в TIFF/JPEG; для PDF используйте OutputIntent, если журнал просит.
- Не конвертируйте RGB → CMYK → RGB многократно: цвета деградируют.

3.4.4 Шрифты в файлах

- PDF: `pdf.fonttype = 42` — TrueType, текст остаётся текстом и встроен.
- SVG: `svg.fonttype = "path"` — текст в кривые, вид стабилен; `"none"` — текст остаётся текстом, но зависит от шрифтов.
- EPS: проверяйте, что шрифты встроены или заменены.
- Перед отправкой откройте PDF в другом просмотрщике и проверьте, что шрифты не подменились.

3.4.5 Метаданные и именование

- Имена файлов — по схеме журнала: `Figure1.pdf`, `fig_01.svg` и т. п.
- Не включайте дату и версию в имя финального файла, если журнал просит иначе; версию храните в репозитории.
- Подпись — отдельным текстом, не внутри изображения.
- Удаляйте дату из SVG-метаданных, если готовите воспроизводимый архив: иначе файл меняется при каждом сохранении.
- Для архивации подходит PDF/A-2b, если журнал его принимает.

3.4.6 Чек-лист форматов

- ☐ Вектор для линий и текста, растр для фото.
- ☐ 300 dpi для фото, 600–1200 dpi для штриховки.
- ☐ Нет JPEG для графиков и текста.
- ☐ Шрифты встроены или переведены в кривые.
- ☐ Цветовой профиль соответствует журналу.
- ☐ Нет скриншотов и апскейла.
- ☐ Имя файла и формат соответствуют требованиям.
- ☐ PDF проверен в другом просмотрщике.

3.5 Сводная таблица требований

Параметр	Значение по умолчанию	Как проверить
Ширина	85 / 114 / 170 мм	`figsize`, PDF при 100 %
Кегль	7–10 pt	линейка в PDF, `fontsize`
Линии	0,25–1 pt	PDF при 100 %
Растр	300 dpi фото, 600+ dpi штрих	размер в пикселях / дюймы
Вектор	PDF/SVG/EPS	текст и линии не пиксельные

Параметр	Значение по умолчанию	Как проверить
Цвет	sRGB онлайн, CMYK печать	профиль, цветопроба
Поля	3–5 мм	PDF, отсутствие обрезки
Контраст	4,5:1 текст, 3:1 графика	WCAG-калькулятор
Подпись	вне изображения, самодостаточная	чтение без текста статьи
Единицы	СИ, пробел перед единицей	вычитка

3.6 Чек-лист раздела 3

- ☐ Палитра соответствует задаче и проверена на дальтонизм.
- ☐ В сером графике сохраняется смысл.
- ☐ Контраст текста и графики достаточен.
- ☐ Шрифт sans-serif, кегли 7–10 pt.
- ☐ Кириллица и специальные символы отображаются.
- ☐ Ширина соответствует журналу; поля не обрезаны.
- ☐ Панели выровнены, метки A/B/C на месте.
- ☐ Вектор/растр выбран по типу содержимого.
- ☐ Разрешение достаточно; нет апскейла и скриншотов.
- ☐ Шрифты встроены; PDF проверен в другом просмотрщике.
- ☐ Цветовой профиль соответствует требованиям.
- ☐ Подпись самодостаточна и не встроена в изображение.

3.7 Итоги

- Цвет — это канал кодирования, а не декор: тип палитры зависит от задачи.
- Качественная палитра — 6–8 цветов, без радуги, с проверкой на CVD.
- Последовательная шкала — монотонная светлота; дивергентная — ноль в центре.
- Шрифт — sans-serif, 7–10 pt в конечном размере, без КАПСА и эффектов.
- Ширина — 85/114/170 мм; проектируйте сразу в конечном размере.
- Вектор для линий и текста, растр для фото; 300/600+ dpi.
- Шрифты встраиваются, профиль проверяется, подпись остаётся текстом.

Дальше — раздел 4: основные виды научных графиков и области их применения. Там мы соберём таблицу «задача → график → ограничения» и перейдём к практике в разделе 5.

Источники к разделу

- Okabe M., Ito K. Color Universal Design (CUD) — jfly.uni-koeln.de/color
- Paul Tol’s colour schemes — personal.sron.nl/ pault
- ColorBrewer 2.0 — colorbrewer2.org
- Viridis colormap — bids.github.io/colormap
- Matplotlib: Choosing colormaps — matplotlib.org
- Machado G. M., Oliveira M. M., Fernandes L. A. F. A Physiologically-based Model for Simulation of Color Vision Deficiency // *IEEE TVCG*. 2009. Vol. 15, № 6. P. 1291–1298. — doi:10.1109/TVCG.2009.113
- W3C. WCAG 2.1: Contrast (Minimum) — w3.org
- W3C. WCAG 2.1: Non-text Contrast — w3.org
- Wilke C. O. *Fundamentals of Data Visualization*. O’Reilly, 2019. — clauswilke.com/dataviz
- Cairo A. *The Truthful Art*. New Riders, 2016. — thefunctionalart.com
- Nature. Formatting guide — nature.com
- ICMJE Recommendations — icmje.org
- ГОСТ 7.32-2017 — docs.cntd.ru
- ГОСТ 8.417-2002 — docs.cntd.ru
- Matplotlib: Fonts — matplotlib.org
- FontTools documentation — fonttools.readthedocs.io

Раздел 4. Основные виды научных графиков и области применения

Выбор графика — первый проектный шаг, и его почти нельзя исправить оформлением. Если для сравнения групп выбран график распределения, читатель не увидит сравнения; если для динамики — столбцы по категориям, он не увидит времени. Этот раздел — **карта**, а не учебник: здесь мы разбираем семейства графиков, их вопросы и ограничения, а подробные разборы с данными, «плохими» и «правильными» версиями будут в разделе 5.

> В разделе 5 каждый **отдельный график** получит свою папку, свой генератор `make_data.py` и свой набор `data/NN_dataset.csv`. Один «вид» на карте — это семейство; внутри него несколько графиков (например, линия, step, area и small multiples), и каждый станет отдельным разделом. Данные одного графика не переиспользуются в другом.

4.1 Пять вопросов перед выбором графика

Полезно задать себе пять вопросов — в этом порядке.

1. **Какой вопрос я задаю?** Сравнить группы, описать распределение, показать связь, динамику, состав, структуру, время до события, качество теста?
2. **Какие переменные?** Категориальные, непрерывные, временные, парные, цензурированные?
3. **Сколько наблюдений и групп?** 10 точек или 10 000? Одна группа или двадцать?
4. **Что важнее читателю?** Точное значение, ранг, форма распределения, неопределённость, наличие выбросов?
5. **Что читатель должен уметь прочитать?** Величину эффекта, разброс, доверительный интервал, порог, число рисков?

Если ответ на первый вопрос не формулируется, график, скорее всего, не нужен: данные лучше показать таблицей.

4.2 Карта семейств

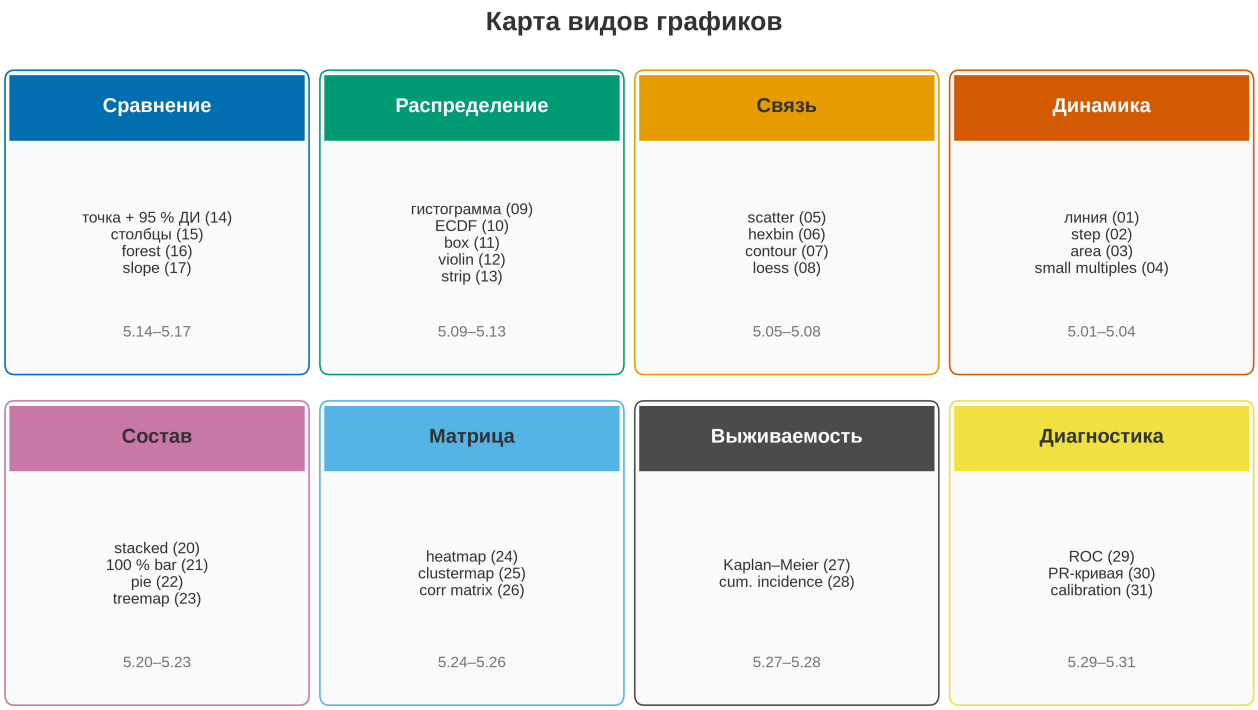


Рисунок 4.1. Карта семейств: вопрос, примеры графиков и номер раздела 5, где вид будет разобран подробно. Карта не заменяет таблицу выбора — она показывает связи между вопросами и формами.

Семейство	Вопрос	Графики (раздел 5)	Ограничения
Сравнение	«Кто больше и насколько?»	точка + 95 % ДИ (5.14), столбцы (5.15), forest (5.16), slope (5.17)	столбцы скрывают распределение; нужна нулевая база
Распределение	«Как устроена выборка?»	гистограмма (5.09), ECDF (5.10), box (5.11), violin (5.12), strip (5.13)	ширина бинов; box скрывает бимодальность; violin требует *n*
Связь	«Есть ли зависимость?»	scatter (5.05), hexbin (5.06), contour (5.07), loess (5.08)	overplotting; корреляция ≠ причинность
Динамика	«Как меняется во времени?»	линия (5.01), step (5.02), area (5.03), small multiples (5.04)	нерегулярные интервалы; нельзя соединять категории
Состав	«Из чего состоит целое?»	stacked bar (5.20), 100 % bar (5.21), pie (5.22), treemap (5.23)	части должны суммироваться; pie — максимум 5 долей
Матрица	«Где значения велики или малы?»	heatmap (5.24), clustermap (5.25), correlation matrix (5.26)	масштаб, кластеризация, colorbar
Выживаемость	«Когда происходит событие?»	Kaplan–Meier (5.27), cumulative incidence (5.28)	цензурирование, at-risk, конкурирующие риски
Диагностика	«Насколько хорош тест?»	ROC (5.29), precision–recall (5.30), calibration (5.31)	prevalence, порог, доверительные интервалы
Парные измерения	«Как меняется у каждого?»	slope chart (5.17), paired dot (5.18), Bland–Altman (5.19)	парность, пропущенные пары
Оценки эффекта	«Какой эффект и насколько он точен?»	forest (5.16), caterpillar (5.32), funnel (5.33)	лог-шкала для отношений, гетерогенность

4.3 Семейства подробнее

4.3.1 Сравнение

Вопрос: «Кто больше, насколько и насколько уверенно?»

Хорошие формы:

- **точка + 95 % ДИ** — лучше столбцов, потому что положение читается точнее длины, а интервал показывает неопределённость;
- **forest plot** — для нескольких оценок и эффектов;
- **slope chart** — для парных измерений;
- **столбцы** — только если нулевая база обязательна и распределение не главное.

Что важно: показывать эффект и ДИ, а не только *p*; не обрезать ось у столбцов; не скрывать *n* и разброс. Подробно — 5.14, 5.15, 5.16, 5.17.

4.3.2 Распределение

Вопрос: «Как устроена выборка: центр, разброс, форма, выбросы?»

Формы:

- **гистограмма** — форма распределения, но зависит от ширины бинов;
- **ECDF** — точное чтение квантилей и сравнение выборок;
- **box** — компактно, но скрывает бимодальность и размер выборки;
- **violin** — форма, но требует достаточного *n* и может вводить в заблуждение при малых группах;
- **strip / swarm** — отдельные точки, лучше при малых *n*;
- **raincloud** — комбинация, но не для каждого журнала.

Подробно — 5.09–5.13.

4.3.3 Связь

Вопрос: «Меняется ли одна величина вместе с другой?»

Формы:

- **scatter** — базовая форма; при больших *n* — **hexbin** или **contour**;
- **loess/regression** — тренд, но не заменяет точки;
- **log-шкалы** — если связь мультипликативная;
- **small multiples** — если есть третья категориальная переменная.

Что важно: не путать корреляцию с причинностью, показывать *n* и выбросы, не использовать две оси Y. Подробно — 5.05–5.08.

4.3.4 Динамика

Вопрос: «Как величина меняется во времени?»

Формы:

- **линия** — регулярные интервалы, одна или несколько серий;
- **step** — измерения «до/после» или дискретные события;
- **area** — накопленный итог, но не для сравнения многих серий;
- **small multiples** — много серий или групп;
- **individual trajectories** — если важна гетерогенность.

Что важно: время — по оси X; не соединяйте линии через категории; показывайте пропуски и нерегулярность; не начинайте ось с произвольного значения без объяснения. Подробно — 5.01–5.04.

4.3.5 Состав

Вопрос: «Из чего состоит целое и как доли меняются?»

Формы:

- **stacked bar** — состав по группам;
- **100 % bar** — сравнение долей;
- **pie** — только 3–5 долей, и только если сумма долей действительно 100 %;
- **treemap** — иерархический состав;
- **alluvial/Sankey** — потоки, но требуют осторожности.

Что важно: части должны суммироваться в целое; круговая диаграмма плохо читается при многих долях и почти всегда проигрывает полосам. Подробно — 5.20–5.23.

4.3.6 Матрица и многомерность

Вопрос: «Где значения велики, где структура, где корреляции?»

Формы:

- **heatmap** — матрица значений; обязательна цветовая шкала с единицами;
- **clustermap** — порядок строк/столбцов подсказывает структуру, но кластеризация субъективна;
- **correlation matrix** — только как дополнение, не как доказательство;
- **PCA/biplot** — уменьшение размерности, но интерпретация меняется;
- **small multiples / parallel coordinates** — альтернатива «радуге».

Подробно — 5.24–5.26.

4.3.7 Выживаемость

Вопрос: «Когда происходит событие и как различаются группы?»

Формы:

- **Kaplan–Meier** — кривые выживаемости;
- **cumulative incidence** — конкурирующие риски;
- **forest plot** — hazard ratio;
- **at-risk table** — число под риском по времени.

Что важно: цензурирование, число рисков, доверительные интервалы, log-rank/Cox, проверка пропорциональности. Кривые без at-risk table и без числа событий читаются плохо. Подробно — 5.27–5.28.

4.3.8 Диагностика

Вопрос: «Насколько хорош тест и при каком пороге?»

Формы:

- **ROC** — чувствительность/специфичность, но зависит от prevalence;
- **precision–recall** — лучше при сильном дисбалансе классов;
- **calibration** — соответствие предсказанных и наблюдаемых вероятностей;
- **decision curve** — клиническая полезность.

Что важно: указывать prevalence, порог, ДИ, число наблюдений; AUROC — не единственная метрика. Подробно — 5.29–5.31.

4.3.9 Парные измерения

Вопрос: «Как меняется каждый объект?»

Формы:

- **slope chart** — линии «до/после» для каждого пациента;
- **paired dot plot** — пары точек;
- **Bland–Altman** — согласие методов, а не корреляция.

Что важно: не анализировать парные данные как независимые группы; показывать пропущенные пары. Подробно — 5.17–5.19.

4.3.10 Оценки эффекта и неопределённость

Вопрос: «Какой эффект и насколько он точен?»

Формы:

- **forest plot** — оценки и ДИ для нескольких групп/исследований;
- **caterpillar** — много оценок;
- **funnel** — публикационное смещение, но только при достаточном числе исследований.

Что важно: эффект + ДИ, лог-шкала для отношений (OR, RR, HR), гетерогенность, а не только *p*. Подробно — 5.16, 5.32–5.33.

4.4 Когда таблица лучше графика

График не всегда выигрывает. Таблица лучше, если:

- значений мало (3–8) и нужна точная выборка;
- единицы разные и график требует нескольких осей;
- важны точные цифры, а не форма;
- категорий слишком много для читаемой палитры;
- данные неполные или несопоставимые.

Практическое правило: **если таблица отвечает на вопрос быстрее и точнее, делайте таблицу**. График — для закономерностей, сравнений и формы, а не для переписывания чисел.

4.5 Антипаттерны выбора

Антипаттерн	Почему плохо	Альтернатива
3D-столбцы и 3D-круг	перспектива искажает длину/площадь	2D
Круговая с 8–12 долями	углы и площади читаются плохо	полосы, stacked bar
Столбцы с SD/SEM	скрывают распределение и *n*	точки, box, violin
Обрезанная ось у столбцов	искажает длину	нулевая база или точки
Линия по категориям	навязывает порядок	столбцы/точки
Две оси Y	ложная корреляция	две панели или индекс
Scatter на 100 000 точек	overplotting	hexbin, density, contour
ROC без prevalence	AUROC не отражает клиническую пользу	PR, calibration
Kaplan–Meier без at-risk	хвост кривой не интерпретируется	at-risk table
Heatmap без шкалы	цвет не читается	colorbar, contour
Radar chart	площадь/угол искажают	small multiples, bar
Word cloud	площадь не пропорциональна	bar
100 % stacked на 20 категорий	невозможно сравнить	small multiples

4.6 Что дальше: как устроен раздел 5

Раздел 5 — практическая часть. Каждый вид графика живёт в своей папке:

```
05_charts/01_line/
├── README.md
├── line.md
├── line.typ
└── scripts/
    ├── make_data.py   # свой генератор → data/01_dataset.csv
    ├── bad_plot.py    # неудачный график → images/05_01_bad_line.*
    └── good_plot.py   # правильный график → images/05_01_good_line.*
```

В каждом разборе будет:

1. сфера применения и типичные вопросы;
2. реалистичные синтетические данные с единицами и диапазонами;
3. неудачный график и разбор ошибок;
4. правильный график и объяснение улучшений;
5. чек-лист и связь с нормативами.

План:

Семейство	**Разделы 5**
Динамика	5.01 линия, 5.02 step, 5.03 area, 5.04 small multiples
Связь	5.05 scatter, 5.06 hexbin, 5.07 contour, 5.08 loess
Распределение	5.09 гистограмма, 5.10 ECDF, 5.11 box, 5.12 violin, 5.13 strip
Сравнение	5.14 точка + ДИ, 5.15 столбцы, 5.16 forest, 5.17 slope
Состав	5.20 stacked bar, 5.21 100 % bar, 5.22 pie, 5.23 treemap
Матрица	5.24 heatmap, 5.25 clustermap, 5.26 correlation matrix
Выживаемость	5.27 Kaplan–Meier, 5.28 cumulative incidence
Диагностика	5.29 ROC, 5.30 precision–recall, 5.31 calibration
Парные	5.17 slope, 5.18 paired dot, 5.19 Bland–Altman
Оценки эффекта	5.16 forest, 5.32 caterpillar, 5.33 funnel

Полный индекс с папками и данными — в `05_charts/README.md`. Порядок может меняться по запросу. Каждый график получит **свой** генератор и **свой** датасет; переиспользование CSV между графиками запрещено.

4.7 Чек-лист выбора графика

- ☐ Вопрос сформулирован одним предложением.
- ☐ Тип переменных определён: категориальные, непрерывные, время, парные, цензурированные.
- ☐ Известны n и число групп.
- ☐ Понятно, что важнее: точное значение, форма, неопределённость.
- ☐ Выбрано семейство, а не «любимый тип графика».
- ☐ Проверены ограничения семейства.
- ☐ Рассмотрена альтернатива в виде таблицы.
- ☐ Цвет и форма не единственные носители смысла.
- ☐ График выдержит конечную ширину и печать.
- ☐ Указан раздел 5, где вид будет разобран подробно.

4.8 Итоги

- Тип графика определяется вопросом и структурой данных, а не привычкой.
- Сравнение, распределение, связь, динамика, состав, матрица, выживаемость, диагностика, парные измерения и оценки эффекта — разные семейства с разными ограничениями.
- Столбцы — не универсальный график; круговая — не универсальный график состава.
- Если таблица точнее и быстрее, выбирайте таблицу.
- Раздел 5 даст полный цикл для каждого из 33 графиков: данные → плохо → разбор → хорошо, с отдельным генератором и датасетом.

Источники к разделу

- Wilke C. O. *Fundamentals of Data Visualization*. O’Reilly, 2019. — clauswilke.com/dataviz
- Cairo A. *The Truthful Art*. New Riders, 2016. — thefunctionalart.com
- Tufte E. R. *The Visual Display of Quantitative Information*. Graphics Press, 1983. — edwardtufte.com
- Cleveland W. S., McGill R. Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods // *JASA*. 1984. Vol. 79, № 387. P. 531–554. — doi:10.1080/01621459.1984.10478080
- Weissgerber T. L. et al. Beyond Bar and Line Graphs: Time for a New Data Presentation Paradigm // *PLoS Biology*. 2015. Vol. 13, № 4. e1002128. — doi:10.1371/journal.pbio.1002128
- From Data to Viz — data-to-viz.com
- Financial Times Visual Vocabulary — github.com/Financial-Times/chart-doctor
- Data Viz Project — datavizproject.com
- CONSORT 2010 — consort-statement.org
- STROBE — strobe-statement.org
- PRISMA — prisma-statement.org

5.01 Линейный график: динамика во времени

Линейный график — самый частый в научных статьях и один из самых обманчивых. Он кажется простым: время по X, значение по Y, точки соединили. Но именно здесь легко спрятать неопределённость, раздуть эффект масштабом и потерять внимание читателя к главному.

Этот раздел — первый практический разбор. Данные, «плохая» и «правильная» версии используют один и тот же CSV; различается только визуализация.

5.01.1 Сфера применения

Линейный график отвечает на вопрос: **как величина меняется во времени или вдоль упорядоченной оси?**

Типичные случаи:

- повторные измерения у одних и тех же пациентов (до/через 4/8/12 недель);
- динамика биомаркеров, АД, ЧСС, глюкозы, HbA1c;
- кривые роста и дозозависимые эффекты;
- временные ряды в эпидемиологии и организации здравоохранения;
- сравнение нескольких групп во времени.

Когда линия **не** подходит:

- ось X — неупорядоченные категории (пол, регион, препарат): линия навязывает порядок и расстояние;
- серий слишком много (больше 5–7): лучше small multiples;
- интервалы времени неравномерны, а вы строите равномерные позиции — это искажает скорость изменения;
- показаны только средние, а разброс и *n* важны для вывода;
- данные цензурированы (выживаемость) — нужен Kaplan–Meier, а не линия.

5.01.2 Данные

Сценарий: гипотетическое 12-недельное исследование, 120 пациентов с артериальной гипертензией, три группы. САД измеряют на 0, 4, 8 и 12-й неделе.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`group`	группа терапии	—
`week`	неделя измерения (0, 4, 8, 12)	недели
`sbp_mm_hg`	систолическое АД	мм рт. ст.

n = 40 в каждой группе, 480 строк в long-формате. Данные синтетические, но правдоподобные по единицам и величинам: плацебо снижает САД примерно до 148, препарат A — до 139, препарат B — до 132 мм рт. ст.

Генератор: `05_charts/01_line/scripts/make_data.py → data/01_dataset.csv`.

5.01.3 Плохой график

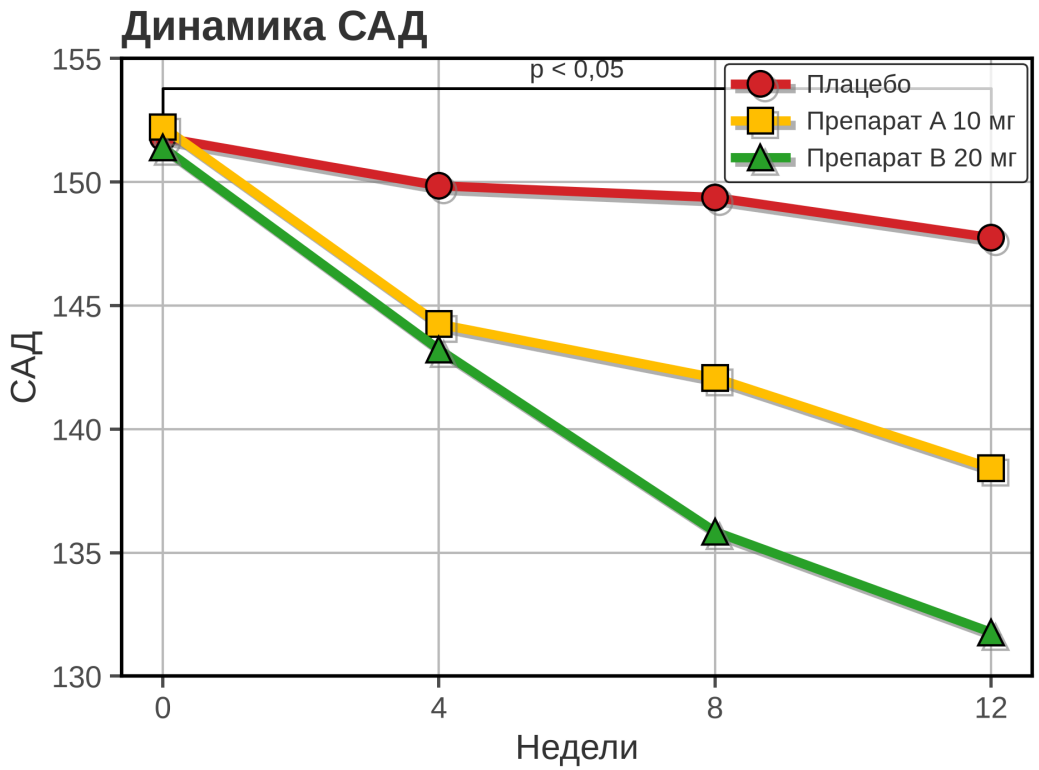


Рисунок 5.1.1. Плохой линейный график: обрезанная ось, светофорные цвета, тени, тяжёлая сетка и легенда поверх данных.

Код: `05_charts/01_line/scripts/bad_plot.py`.

Что с ним не так:

1. **Обрезанная ось Y (130–155).** Для линии нулевая база не обязательна, но диапазон не должен обрезать данные и создавать ложный драматизм. Здесь препарат В почти упирается в нижнюю границу, а плацебо выглядит почти плоским; читатель переоценивает разницу.
2. **Светофорная палитра.** Красный — жёлтый — зелёный неразличим при протанопии/дейтеранопии и навязывает оценку «плохо — средне — хорошо».
3. **Толстые линии, тени и крупные маркеры.** Тень не кодирует данные; она утяжеляет линии и мешает сравнивать наклоны.
4. **Тяжёлая сетка по обеим осям и полная рамка.** Сетка конкурирует с данными, вертикальные линии не помогают читать недели.
5. **Заголовок «Динамика САД».** Он не сообщает, что именно произошло и какой вывод должен сделать читатель.
6. **Нет единиц, *n* и неопределённости.** «САД» — это что: исходное, конечное, среднее? В каких единицах? Сколько пациентов?
7. **Только средние.** Без доверительных интервалов или полос разброса нельзя понять, насколько устойчива разница.
8. **Легенда внутри поля данных.** Она дублирует подписи и закрывает часть линий.
9. **Брекет «p < 0,05» без теста и эффекта.** Значимость не заменяет величину эффекта; не указано, какой тест и какие группы сравнивались.

Дополнительно: равномерные позиции 0/4/8/12 здесь корректны, потому что интервалы равны. Если бы интервалы были неравными (0, 2, 12, 52 недели), точки нужно ставить по реальному времени, а не через равные шаги.

5.01.4 Правильный график

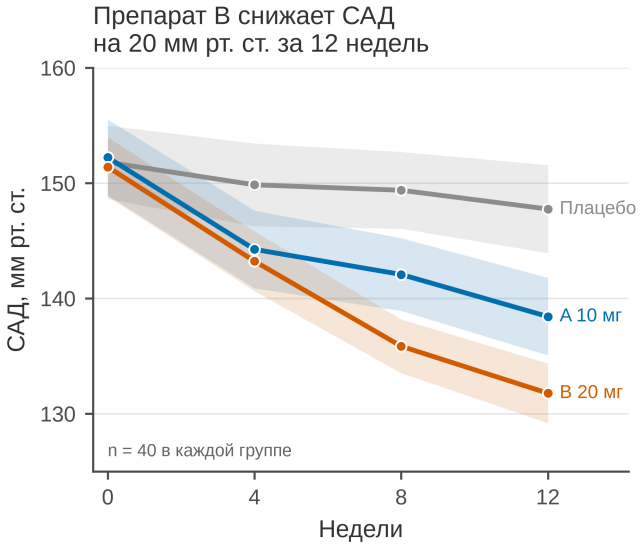


Рисунок 5.1.2. Правильный линейный график: среднее и 95 % ДИ, палитра Okabe–Ito, прямые подписи и заголовок-вывод.

Код: `05_charts/01_line/scripts/good_plot.py`.

Что улучшено:

1. **Ширина 85 мм, высота 72 мм.** Рисунок спроектирован в конечном журнальном размере, а не уменьшен из слайда.
2. **Среднее и 95 % ДИ.** Полосы показывают неопределённость; видно, где различия уверенные, а где интервалы пересекаются.
3. **Палитра Okabe–Ito.** Серый — контроль, синий — препарат A, вермилион — препарат B; цвета различимы при дальтонизме.
4. **Прямые подписи в конце линий.** Легенда не нужна; читатель сразу видит, какая линия какой группе соответствует.
5. **Заголовок-вывод.** «Препарат B снижает САД на 20 мм рт. ст. за 12 недель» — сразу понятно, что читать.
6. **Единицы и *n*.** «САД, мм рт. ст.» и «n = 40 в каждой группе»; в подписи можно добавить определение ДИ.
7. **Ось Y 125–160.** Достаточный контекст: видны исходный уровень, конечные значения и величина снижения, без обрезки «под эффект».
8. **Лёгкая сетка только по Y.** Верхняя и правая рамки убраны; данные контрастнее осей.
9. **Маркеры только в измеренных точках.** Линия соединяет реальные недели 0/4/8/12, а не произвольные позиции.

5.01.5 Сопоставление

Что было	Почему плохо	Что сделано
ось Y 130–155	масштаб раздувает различия	125–160, виден контекст
красный/жёлтый/зелёный	ложная семантика, CVD	Okabe–Ito: серый/синий/вермилион
тени, толстые линии	декор, мешает сравнивать наклоны	линии 1,8 pt, маркеры 4 pt
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
легенда	дублирует подписи	прямые подписи
«Динамика САД»	нет сообщения	заголовок-вывод
нет единиц и *n*	невозможно прочитать	единицы, *n*
только средние	скрыта неопределённость	среднее + 95 % ДИ
p < 0,05	нет теста и эффекта	эффект и ДИ; тест — в подписи

5.01.6 Чек-лист линейного графика

- ☐ Время (или упорядоченная ось) — по X, значения — по Y.

- ☐ Интервалы на оси X соответствуют реальному времени.
- ☐ Ось Y не обрезает данные и не создаёт ложного драматизма.
- ☐ Показаны среднее и 95 % ДИ (или individual trajectories).
- ☐ Указаны *n* и единицы.
- ☐ Цвет не единственный носитель смысла.
- ☐ Легенда не закрывает данные; лучше прямые подписи.
- ☐ Заголовок сообщает вывод, а не «динамику».
- ☐ Нет 3D, теней и тяжёлой сетки.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.01.7 Как воспроизвести

```
uv sync
uv run python 05_charts/01_line/scripts/make_data.py
uv run python 05_charts/01_line/scripts/bad_plot.py
uv run python 05_charts/01_line/scripts/good_plot.py
```

Ключевой фрагмент «правильного» кода:

```
fig, ax = plt.subplots(
    figsize=figure_size("single", height_mm=72),
    layout="constrained",
)
ax.fill_between(x, part["ci_low"], part["ci_high"], color=color, alpha=0.15)
ax.plot(x, part["mean"], color=color, marker="o", linewidth=1.8)
ax.annotate(GROUP_SHORT[group], (x[-1] + 0.08, part["mean"].iloc[-1]))
save_figure(fig, "05_01_good_line")
```

5.01.8 Что дальше

Следующий график — **5.02 step**: ступенчатая функция для событий, измерений «до/после» и дискретных изменений. У него будет свой генератор и свой `data/02_dataset.csv`.

Источники

- Cleveland W. S., McGill R. Graphical Perception // *JASA*. 1984. — doi:10.1080/01621459.1984.10478080
- Weissgerber T. L. et al. Beyond Bar and Line Graphs // *PLoS Biology*. 2015. — doi:10.1371/journal.pbio.1002128
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz

5.02 Ступенчатый график: события и дискретные изменения

Ступенчатый график — это линия, у которой значение **сохраняется** между событиями и меняется скачком в момент события. Он нужен там, где процесс по своей природе дискретен: доза препарата, скорость инфузии, статус, тариф, число коек, порог.

Главная ошибка — соединить такие точки наклонной линией. Тогда читатель видит плавное изменение, которого в данных не было.

5.02.1 Сфера применения

Ступенчатая функция отвечает на вопрос: **в какой момент значение изменилось и каким оно оставалось до следующего события?**

Типичные случаи:

- титрование дозы препарата (варфарин, инсулин, антибиотик);
- скорость инфузии (мкг/кг/мин, мл/ч);
- изменение статуса (госпитализирован/выписан, жив/умер — но для выживаемости нужен Kaplan–Meier);
- тарифы, оклады, налоговые ставки;
- число занятых коек или доступных аппаратов;
- ступенчатые протоколы и пороговые правила.

Когда ступенчатый график **не** подходит:

- величина меняется непрерывно (концентрация, температура, АД);
- ось X — неупорядоченные категории;
- нужно показать распределение или неопределённость, а не момент изменения;
- данных слишком мало для шага — тогда лучше точки.

5.02.2 Данные

Сценарий: три пациента на варфарине, 12 недель. Дозу меняют на визитах раз в неделю, шагом 0,5 мг/сут; INR измеряют на каждом визите, целевой диапазон — 2,0–3,0.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`week`	неделя визита (0–12)	недели
`dose_mg`	доза варфарина	мг/сут
`inr`	международное нормализованное отношение	—

39 строк: 3 пациента × 13 визитов. Дозы — 3,0–7,5 мг/сут; у P01 доза стабильна, у P02 и P03 — титруется. Это не исследование, а иллюстрация индивидуального титрования; для групповых выводов нужны другие графики и статистика.

Генератор: `05_charts/02_step/scripts/make_data.py → data/02_dataset.csv`.

5.02.3 Плохой график

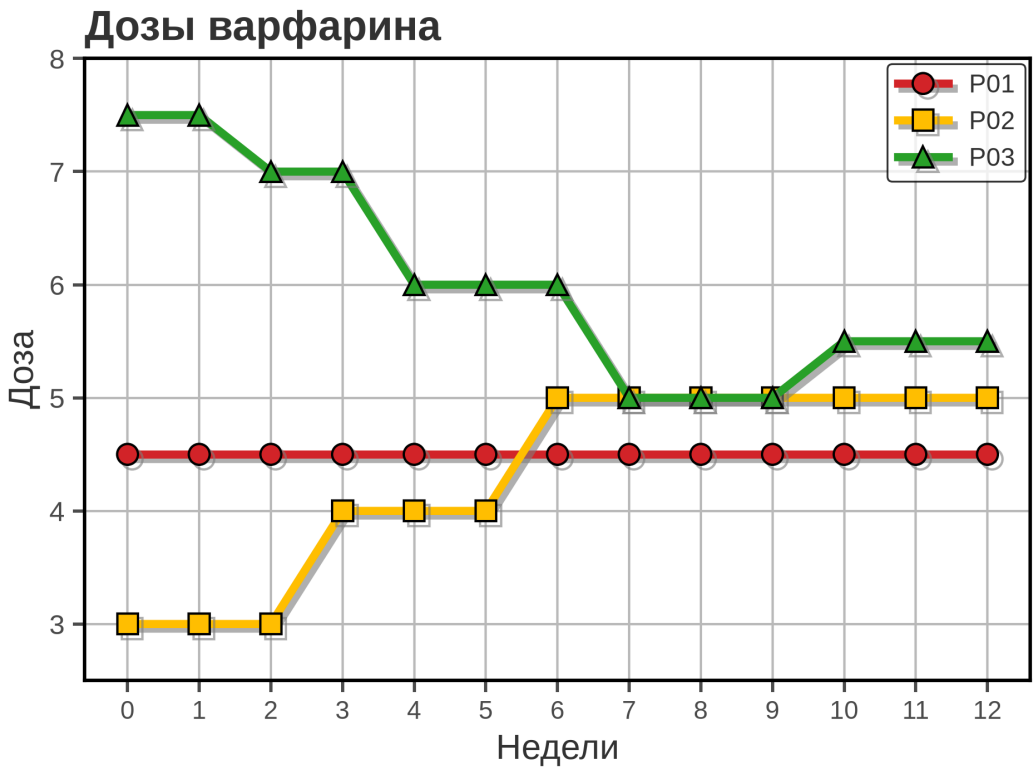


Рисунок 5.2.1. Плохой ступенчатый график: наклонные линии вместо ступеней, обрезанная ось, светофорные цвета и тени.

Код: `05_charts/02_step/scripts/bad_plot.py`.

Что с ним не так:

1. **Наклонные линии вместо ступеней.** Доза не меняется плавно между визитами; `ax.plot` создаёт ложное впечатление непрерывного титрования. Это главная ошибка.
2. **Обрезанная ось Y (2,5–8).** Масштаб раздувает различия; нулевая база для дозы не обязательна, но диапазон должен показывать реальный масштаб.
3. **Светофорная палитра.** Красный — жёлтый — зелёный неразличим при протанопии/дейтеранопии и навязывает оценку «плохо — средне — хорошо».
4. **Толстые линии, тени и крупные маркеры.** Декор мешает видеть моменты изменения.
5. **Тяжёлая сетка по обеим осям и полная рамка.** Шум, который конкурирует с данными.
6. **Заголовок «Дозы варфарина».** Нет сообщения: что происходит с дозами и почему это важно.
7. **Нет единиц, числа пациентов и контекста INR.** «Доза» — в чём? Сколько пациентов? Почему дозу меняли?
8. **Легенда внутри поля данных.** Она дублирует подписи и закрывает линии.

5.02.4 Правильный график

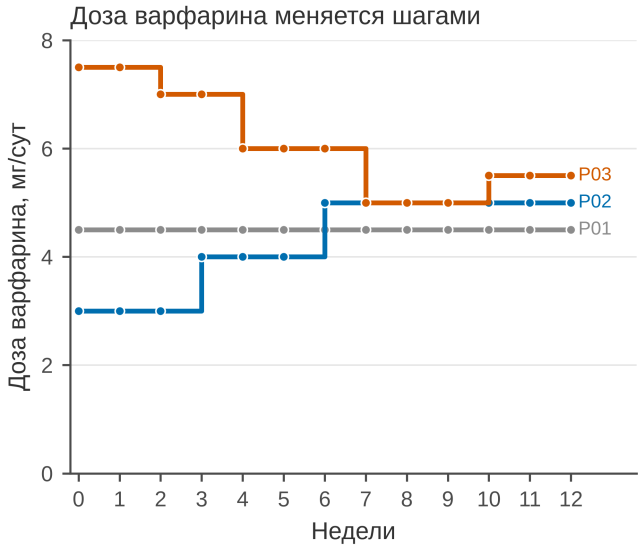


Рисунок 5.2.2. Правильный ступенчатый график: steps-post, маркеры визитов, прямые подписи и ось 0–8 мг/сут.

Код: `05_charts/02_step/scripts/good_plot.py`.

Что улучшено:

1. **`ax.step(..., where="post")`.** Доза сохраняется до следующего визита; вертикальные сегменты показывают скачок, горизонтальные — период постоянной дозы.
2. **Маркеры в моменты визитов.** Видно, где именно измеряли и меняли дозу, а не абстрактную кривую.
3. **Прямые подписи P01/P02/P03.** Легенда не нужна; конец линии сразу подписан.
4. **Ось Y 0–8 мг/сут.** Виден реальный масштаб доз, без обрезки «под эффект».
5. **Единицы и недели.** «Доза варфарина, мг/сут», недели 0–12.
6. **Заголовок-вывод.** «Доза варфарина меняется шагами» объясняет, что читать.
7. **Палитра Okabe–Ito.** Серый — P01, синий — P02, вермилион — P03.
8. **Лёгкая сетка только по Y.** Верхняя и правая рамки убраны.
9. **85 мм в ширину.** Рисунок спроектирован в конечном размере.

INR не вынесен на вторую ось: две оси Y создают ложную корреляцию. Если INR важен, сделайте отдельную панель или таблицу; в подписи укажите целевой диапазон 2,0–3,0.

5.02.5 Сопоставление

Что было	Почему плохо	Что сделано
`ax.plot`	плавное изменение, которого не было	`ax.step(where="post")`
ось Y 2,5–8	масштаб раздувает различия	0–8 мг/сут
красный/жёлтый/зелёный	ложная семантика, CVD	Okabe–Ito
тени, толстые линии	декор	линии 1,8 pt, маркеры 3,5 pt
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
легенда	дублирует подписи	прямые подписи
«Дозы варфарина»	нет сообщения	заголовок-вывод
нет единиц и *n*	невозможно прочитать	единицы, число пациентов в подписи
INR на второй оси	ложная корреляция	INR в подписи/отдельной панели

5.02.6 Чек-лист ступенчатого графика

- ☐ Величина действительно постоянна между событиями.
- ☐ Использован `step`, а не `plot` с наклонными линиями.
- ☐ Выбрано `where="post"` или `where="pre"` осознанно.
- ☐ Маркеры стоят в моменты измерения/изменения.
- ☐ Ось Y показывает реальный масштаб, без обрезки «под эффект».
- ☐ Указаны единицы, число пациентов и контекст.

- ☐ Прямые подписи лучше легенды.
- ☐ Нет двух осей Y.
- ☐ Цвет не единственный носитель смысла.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.02.7 Как воспроизвести

```
uv sync
uv run python 05_charts/02_step/scripts/make_data.py
uv run python 05_charts/02_step/scripts/bad_plot.py
uv run python 05_charts/02_step/scripts/good_plot.py
```

Ключевой фрагмент:

```
ax.step(
    weeks,
    doses[patient].to_numpy(),
    where="post",
    color=color,
    linewidth=1.8,
    marker="o",
    markersize=3.5,
)
```

`where="post"` означает: значение `y[i]` держится от `x[i]` до `x[i+1]`, затем происходит скачок. Если событие фиксируется в конце интервала, проверьте, не нужен ли `where="pre"`.

5.02.8 Что дальше

Следующий график — **5.03 area**: область под линией для накопленных и составных величин. У него будет свой генератор и свой `data/03_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `step` — matplotlib.org
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com

5.03 Area chart: площадь как величина

Area chart — это линия с заливкой под ней. Он оправдан только тогда, когда **площадь является смыслом**: площадь под кривой концентрации (AUC), накопленная экспозиция, суммарная нагрузка, cumulative burden. Если заливка просто «украшает» линию, она добавляет чернила, но не информацию.

Главное правило: **заливка area chart должна начинаться с нуля**. Иначе площадь перестаёт быть пропорциональна величине, которую она якобы кодирует.

5.03.1 Сфера применения

Area chart отвечает на вопрос: **какая суммарная величина накоплена за период?**

Типичные случаи:

- площадь под кривой концентрация–время (AUC) в фармакокинетике;
- накопленная экспозиция лекарства или излучения;
- суммарная нагрузка (число случаев, койко-дни, затраты) за период;
- cumulative incidence, если это не кривая выживаемости (для выживаемости — Kaplan–Meier, 5.27);
- stacked area для состава во времени, но с осторожностью: наложение областей затрудняет сравнение (для состава чаще лучше stacked bar, 5.20).

Когда area chart **не** подходит:

- площадь не несёт смысла, а заливка нужна «для красоты» — используйте линию;
- несколько перекрывающихся областей: одна заливка закрывает другую;
- ось Y не начинается с нуля;
- нужно точное сравнение значений — линия или точки читаются лучше;
- категории по X не упорядочены.

5.03.2 Данные

Сценарий: 40 пациентов получают условный препарат внутрь в дозе 500 мг. Концентрацию в плазме измеряют через 0, 1, 2, 4, 6, 8, 12 и 24 часа. AUC 0–24 — интегральная экспозиция, мг·ч/л.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`time_h`	время после приёма	часы
`conc_mg_l`	концентрация в плазме	мг/л

320 строк: 40 пациентов × 8 моментов. Средний AUC 0–24 — 80,7 ± 18,2 мг·ч/л; пик концентрации — около 8–9 мг/л через 1–2 часа; к 24 часам — около 0,5 мг/л.

Генератор: `05_charts/03_area/scripts/make_data.py → data/03_dataset.csv`.

5.03.3 Плохой график

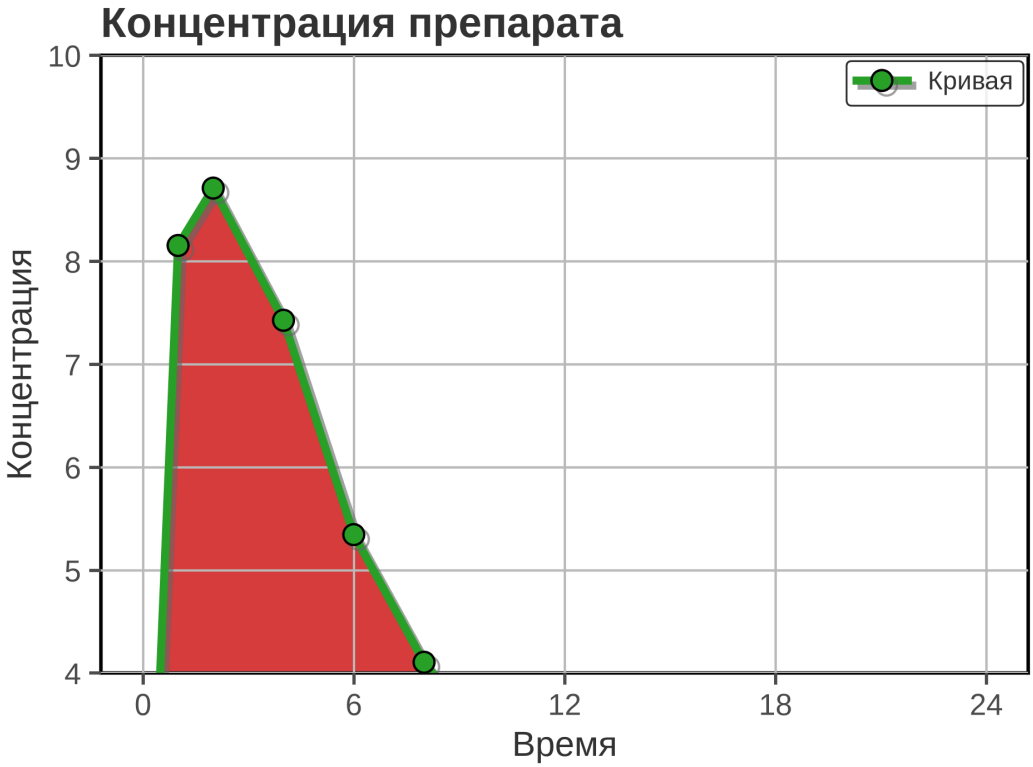


Рисунок 5.3.1. Плохой area chart: заливка от 4 вместо нуля, обрезанная ось, красная заливка и зелёная линия, тени, тяжёлая сетка.

Код: `05_charts/03_area/scripts/bad_plot.py`.

Что с ним не так:

1. **Заливка начинается с y = 4, а не с нуля.** Площадь больше не пропорциональна AUC; график показывает не интеграл, а обрезок. Это главная ошибка.
2. **Обрезанная ось Y (4–10).** Пик и хвост искажены; нижняя часть кривой и базовая линия скрыты.
3. **Красная заливка и зелёная линия.** Светофорная пара неразличима при протанопии/дейтеранопии; тёмная заливка конкурирует с линией.
4. **Тени и крупные маркеры.** Декор, который не кодирует данные.
5. **Тяжёлая сетка по обеим осям и полная рамка.** Сетка просвечивает через заливку и создаёт шум.
6. **Заголовок «Концентрация препарата».** Не сообщает, что площадь — это экспозиция и какова её величина.
7. **Нет единиц, *n* и AUC.** «Концентрация» — в чём? Сколько пациентов? Чему равна площадь?
8. **Легенда из одной строки «Кривая».** Она не несёт информации.

5.03.4 Правильный график

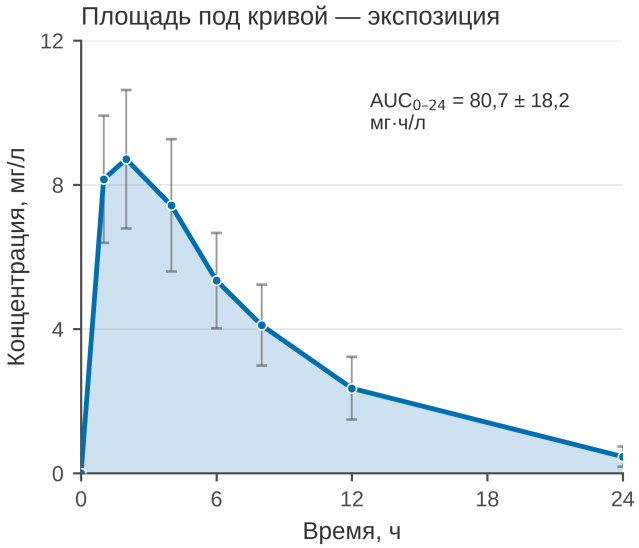


Рисунок 5.3.2. Правильный area chart: заливка от нуля, подпись AUC, среднее ± SD, палитра Okabe–Ito, ось 0–12 мг/л.

Код: `05_charts/03_area/scripts/good_plot.py`.

Что улучшено:

1. **Заливка от 0.** Площадь под кривой действительно равна AUC; базовая линия видна.
2. **Подпись AUC 0–24 = 80,7 ± 18,2 мг·ч/л.** Площадь превращается в число, которое можно интерпретировать.
3. **Ось Y 0–12 мг/л.** Видны пик, хвост и нулевая база; нет обрезки «под эффект».
4. **Среднее ± SD в точках.** Видна неопределённость; заливка показывает интеграл, а не заменяет разброс.
5. **Палитра Okabe–Ito.** Один синий цвет для одной серии; ничего лишнего.
6. **Единицы и *n*.** «Концентрация, мг/л», «Время, ч», *n* = 40 в подписи.
7. **Заголовок-вывод.** «Площадь под кривой — экспозиция» объясняет, зачем здесь заливка.
8. **Лёгкая сетка только по Y, без верхней и правой рамок.**
9. **85 мм в ширину.** Рисунок спроектирован в конечном размере.

Если площадь не нужна, эти же данные лучше показать линией без заливки: меньше чернил, та же информация. Area chart — не «линия с красивым фоном», а график интеграла.

5.03.5 Сопоставление

Что было	Почему плохо	Что сделано
заливка от 4	площадь не равна AUC	заливка от 0
ось Y 4–10	обрезаны пик и хвост	0–12 мг/л
красная заливка + зелёная линия	CVD, шум	один синий цвет
тени, крупные маркеры	декор	линия 1,8 pt, маркеры 3,5 pt
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
легенда «Кривая»	бесполезна	без легенды
«Концентрация препарата»	нет сообщения	заголовок-вывод
нет AUC и единиц	площадь не читается	AUC, мг·ч/л, *n*
только среднее	нет неопределённости	среднее ± SD

5.03.6 Чек-лист area chart

- ☐ Площадь действительно является смыслом (AUC, накопление).
- ☐ Заливка начинается с нуля.
- ☐ Ось Y не обрезает нижнюю часть площади.
- ☐ Подписана величина площади (AUC, сумма, интеграл).

- ☐ Показана неопределённость (SD/CI или отдельные кривые).
- ☐ Указаны единицы, *n*, время.
- ☐ Нет перекрывающихся непрозрачных областей.
- ☐ Заливка не конкурирует с линией и сеткой.
- ☐ Если площадь не нужна — выбран line chart.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.03.7 Как воспроизвести

```
uv sync
uv run python 05_charts/03_area/scripts/make_data.py
uv run python 05_charts/03_area/scripts/bad_plot.py
uv run python 05_charts/03_area/scripts/good_plot.py
```

Ключевые фрагменты:

```
ax.fill_between(time, 0, mean, color=PRIMARY_BLUE, alpha=0.18, linewidth=0)
ax.plot(time, mean, color=PRIMARY_BLUE, marker="o", linewidth=1.8)

auc = np.trapezoid(part["conc_mg_l"], part["time_h"])
```

`np.trapezoid` считает площадь трапеций; единицы AUC — мг·ч/л, потому что концентрация в мг/л умножается на время в часах.

5.03.8 Что дальше

Следующий график — **5.04 small multiples**: несколько панелей с общей шкалой для многомерных данных. У него будет свой генератор и свой `data/04_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `fill_between` — matplotlib.org
- NumPy: `trapezoid` — numpy.org
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com